



**COMSATS University Islamabad,
Sahiwal Campus.**

Event Sphere

By

Mashhad Jamal

CIIT/FA22-BCS-074/SWL

Muneeb Tahir

CIIT/ FA22-BCS-076/SWL

M Noman Zafar

CIIT/ FA22-BCS-078/SWL

Supervisor

M Ahmad Farid &

Co Supervised by

Ali Shair Kashif

Bachelor of Science in Computer Science (2022-2026)



**COMSATS University Islamabad,
Sahiwal Campus.**

Event Sphere

A Project Presented To

**COMSATS University Islamabad,
Sahiwal Campus**

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Mashhad Jamal

CIIT/FA22-BCS-074/SWL

Muneeb Tahir

CIIT/ FA22-BCS-076/SWL

M Noman Zafar

CIIT/ FA22-BCS-078/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied the report entirely based on our efforts. If any part of this project is proved to be copied from any source or found to be the reproduction of some other. We will stand by the consequences. No portion of the work presented has been submitted because of any application for any other degree or qualification of this or any other university or institute of learning.

Mashhad Jamal

Muneeb Tahir

M Noman Zafar

CERTIFICATE OF APPROVAL

This is to certify that the final year project of BS(CS) “**Event Sphere**” was developed by Mashhad Jamal (CIIT/FA22-BCS-074/SWL), Muneeb Tahir (CIIT/FA22-BCS-076/SWL) and M Noman Zafar (CIIT/FA22-BCS-078/SWL) under the supervision of “**M Ahmad Farid**” and co supervisor “**Ali Shair Kashif**” and that in his opinion; it is fully adequate, in scope and quality for the degree of bachelors of science in Computer Science.

Supervisor

M Ahmad Farid

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

Universities host a wide array of academic and extracurricular events throughout the year, yet most institutions still rely on fragmented, manual processes for management and communication. This leads to low student engagement, inefficient attendance tracking, and a lack of centralized data for administrative oversight. Traditional methods— such as physical notice boards, informal WhatsApp groups, and manual sign-in sheets—are often error-prone, time- consuming, and fail to provide a cohesive experience for stakeholders.

To bridge these gaps, Event Sphere was developed. It is a digital, role-based event management system designed to centralize and automate the entire event lifecycle within a university campus. The application provides a unified platform where Faculty can propose events, Administrators can review and approve them, and Students can discover and register for events seamlessly through their mobile devices.

Event Sphere replaces outdated manual workflows with intelligent features, including an automated QR code-based attendance system that prevents duplicate check-ins and human error. It also incorporates real-time push notifications, a dedicated expense management module for financial accountability, and an AI-powered assistant to provide instant support to users.

Technically, the system is a cross-platform mobile application built using the Flutter framework and Dart. It leverages a robust cloud-backend powered by Firebase (Firestore, Authentication, and Storage) and Supabase to ensure real- time data synchronization, scalability, and secure role-based access control. By digitizing event coordination, Event Sphere enhances campus engagement and provides administrators with the analytics needed for data-driven decision-making.

Acknowledgement

All praise is to the almighty Allah, who bestowed upon us a minute portion of his boundless knowledge by which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “**M Ahmad Farid**” and our Co- Supervisor “**Ali Shair Kashif**”. Without their supervision, advice, and valuable guidance, the completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

We are also thankful to our parents, family and friends, and **Osama bin Jamil** who have been a constant source of encouragement for us and have brought us the values of honesty & hard work.

Mashhad Jamal

Muneeb Tahir

M Noman Zafar

Abbreviations

SRS	Software Require Specification
PC	Personal Computer
SDD	Software Design Description
FYP	Final Year Project
API	Application Programming Interface
UI	User Interface
UX	User Experience
DB	Database
FR	Functional Requirement
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
FCM	Firebase Cloud Messaging
QR	Quick Response (Code)
OOP	Object-Oriented Programming
OOD	Object-Oriented Design
DFD	Data Flow Diagram
UML	Unified Modeling Language
CRUD	Create, Read, Update, Delete
NoSQL	Non-Relational Database
APK	Android Package Kit
AI	Artificial Intelligence
SDK	Software Development Kit

Table of Contents

Chapter 1 Introduction

1	Introduction	2
1.1	Brief Overview	2
1.2	Relevance to Course Modules	3
1.3	Project Background	4
1.4	Literature Review	5
1.5	Analysis from Literature Review	6
1.6	Methodology and Software Lifecycle for This Project	6
	System Design:.....	7
2	Problem Definition	12
2.1	Problem Statement.....	12
2.2	Development Requirements.....	14
2.2.1	Functional Requirements.....	15
2.2.2	Non-Functional Requirements:	15
2.3	Current System	16
3	Requirement Analysis	19
3.1	Use Case Diagrams.....	19
3.1.1	Actors	20
3.1.2	Primary Use Cases:	20
3.1.3	Use Case Relationships	21
3.2	Detailed Use Case Descriptions.....	21
3.2.1	Use Case Summary	21
3.3	Functional Requirements	22
3.3.1	User Registration and Login	22
	<i>Table 3.1: Functional Requirements — User Registration and Login.....</i>	<i>22</i>
3.3.2	Event Creation and Management	23
3.3.3	Event Registraion and Participation	23
3.3.4	QR Code Scanning and Attendance	24
3.3.5	Notifications	24
3.3.6	Reports and Analytics	24
3.3.7	Expense (Cost) Management.....	25

3.3.8	System Administration and Maintenance	25
3.4	Non-Functional Requirements	26
3.4.1	Usability Requirements	26
3.4.2	Performance Requirements.....	26
3.4.3	Scalability Requirements.....	27
3.4.4	Availability Requirements.....	27
3.4.5	Compatibility Requirements.....	28
3.4.6	Maintainability Requirements	28
4	Design and Architecture	30
4.1	System Architecture.....	30
4.1.1	Presentation Layer.....	30
4.1.2	Application Layer	31
4.1.3	Data Layer	33
4.2	Data Representation.....	34
4.2.1	Data Dictionary	34
4.2.2	Entity Relationships	36
4.2.3	ER Diagram Overview	37
4.3	Process Flow / Representation.....	38
4.3.1	Activity Diagram.....	38
4.3.2	General Workflow	39
4.3.3	Design Models.....	40
4.3.4	Sequence Diagram.....	41
4.4	Class Diagram	42
4.5	Data Flow Diagrams	43
4.6	DFD Level 1 — Event Management	44
5	Implementation.....	46
5.1	Algorithm	46
5.2	External APIs.....	50
5.3	User Interface	52
5.4	Database Security - Row Level Security	60
5.4.1	Environment ConFigureuration and API Key Management	60
5.4.2	State Management and Architecture Patterns.....	60
5.4.3	Expanded Role-Based Access Control System.....	61

5.4.4	Gamification Module	61
5.4.5	AI Chatbot Assistant	61
6	Testing and Evaluation	64
6.1	Testing Strategy	64
6.1.1	Testing Approach Summary.....	64
6.2	Test Plan	65
6.3	Test Environment.....	66
6.3.1	Hardware	66
6.3.2	Software	67
6.4	Test Cases	67
6.4.1	Authentication Test Cases.....	67
6.4.2	Event Management Test Cases	68
6.4.3	Event Registration Test Cases	69
6.4.4	QR Code Attendance Test Cases.....	70
6.4.5	Notification Test Cases.....	71
6.4.6	Expense Management Test Cases.....	72
6.4.7	Admin Dashboard Test Cases.....	73
6.4.8	Performance Test Cases.....	74
6.5	Evaluation Results.....	75
6.5.1	Functional Results	75
6.5.2	Performance Results.....	75
6.5.3	Usability Results	76
6.5.4	Robustness.....	76
6.6	Discussion.....	76
7	Conclusion and Future Work.....	78
7.1	Conclusion	78
7.2	Limitatins.....	79
7.3	Future Improvements.....	80

List of Figures

Figure 1.1: Agile Model Diagram	6
Figure 3.1: Use Case Diagram.....	19
Figure 4.1: Activity Diagram.....	38
Figure 4.2: Sequence Diagram	41
Figure 4.3: Class Diagram	42
Figure 4.4: Data Flow Diagram	43
Figure 4.5: DFD Level 1 Event Management.....	44
Figure 5.1: Splash Screen	54
Figure 5.2: Onboarding Screen.....	54
Figure 5.3: Login Screen	55
Figure 5.4: Sign Up Screen.....	55
Figure 5.5: Home Page	56
Figure 5.6: Profile Page	56
Figure 5.7: Event List Page	57
Figure 5.8: Event Detail Page.....	57
Figure 5.9: Create Event Page	58
Figure 5.10: QR Code Scanner Page	58
Figure 5.11: Event Registration Page	59
Figure 5.12: Admin Event Approval Page	59
Figure 5.13: Expense Management Page.....	60
Figure 5.14: Report and Analytics.....	60

List of Tables

Table 1.1: Relevance to Course Modules	3
Table 2.1: StakeHolder Specific Challenges	13
Table 2.2: Current System vs. Event Sphere	16
Table 3.1: Functional Requirements	22
Table 3.2: Functional Requirements	23
Table 3.3: Functional Requirements	23
Table 3.4: Functional Requirements	24
Table 3.5: Functional Requirements	24
Table 3.6: Functional Requirements	24
Table 3.7: Functional Requirements	25
Table 3.8: Functional Requirements	25
Table 3.9: Usability Requirements	26
Table 3.10: Performance Requirements	26
Table 3.11: Scalability Requirements	27
Table 3.12: Availability Requirements	27
Table 3.13: Compatibility Requirements	28
Table 3.14: Maintainability Requirements	28
Table 4.1: Data Dictionary — Event Sphere	34
Table 4.2: Entity Relationship Specification	37
Table 5.1: External Libraries and APIs	51
Table 6.1: Test Plan — Event Sphere	65
Table 6.2: Authentication Test Cases	67
Table 6.3: Event Management Test Cases	68
Table 6.4: Event Registration Test Cases	68
Table 6.5: QR Code Attendance Test Cases	69
Table 6.6: Notification Test Cases	70
Table 6.7: Expense Management Test Cases	71
Table 6.8: Admin Dashboard Test Cases	72
Table 6.9: Performance Test Cases	73
Table 6.10: Performance Evaluation Results	74

Chapter 1

Introduction

1 Introduction

The advent of mobile and cloud computing technologies has revolutionised information management and dissemination in organisations. Universities have a massive number of events held throughout the academic year ranging from seminars, conferences, sports, cultural, and other departmental events. Despite this, nearly all universities still use disjointed and offline approaches for event communication and coordination, resulting in low student participation, low turnout and wasted time and money. Event Sphere is an Android mobile application that aims to solve these problems by offering a mobile digital event management system for university events. It supports Faculty for proposing events, which are submitted for review and approved by Administrators before going public to Students. Students can view all events and sign up for events they are interested in, using a QR code scanning system to verify attendance. Event Sphere also enables real-time notifications, attendance tracking, and role-based authorization for Administrators, Faculty and Students. Event Sphere is backed by Firebase and Supabase, which enable real-time data updates, secure authentication, and cloud data storage. Event locations are shown using Google Maps API and a marketing web page allows general overview of the application and APK downloading. The application is built with Flutter and Dart, offering a consistent look and feel and optimal performance on Android devices. This report offers a detailed description of the Event Sphere design, implementation and testing. It starts with the problem context and background of the project, followed by a detailed review of the requirements based on the Software Requirements Specification (SRS) document. The following chapters address the system design and architecture (Software Design Description - SDD), implementation, testing, and conclusions.

1.1 Brief Overview

Event Sphere is a mobile app for event management in universities to connect users with administrators, faculty and students to handle university events. It is designed for administrators to have complete oversight of all user accounts, events and system management. It gives faculty the ability to submit, manage, and track events, and students with the ease of discovering, registering and scanning QR codes at their events. Flutter, a mobile app development framework, has been used to develop the project, allowing it to run on Android. Real-time database operations and user authentication are managed through Firebase, with Supabase offering additional back-end services. It uses Google Maps API for displaying the events' location and has an automated notification module to ensure that users are always up to date with changes and notifications. Our approach for development is Agile and we use sprints. The sprints

were dedicated to the development of these modules: login and registration, event creation, QR attendance system, notifications, and admin dashboard. This provided feedback throughout the project and, eventually, functional software at the end of each sprint.

1.2 Relevance to Course Modules

Event Sphere development incorporates theoretical concepts and hands-on skills learned in several courses of the Bachelor of Science in Computer Science (BSCS) program. Table 1.1 below shows the relevance of each course module to the development of Event Sphere.

Table 1.1: Relevance to Course Modules

Course Module	Application in Event Sphere
Software Engineering	Sphere adopted a Software Engineering development approach. Software Requirement Specification (SRS) and System Design Document (SDD) were developed to specify the system requirements and design. Use case, class, sequence and activity diagrams were created to describe system behavior. The Agile SDLC methodology was used to enable iterative development and incremental feature release.
Mobile-Application Development	System was developed as a mobile application using the cross- platform framework Flutter and Dart. A user interface has been built on Android devices, allowing all five user roles to access a mobile responsive design. Event browsing, event registration, scanning QRs and dealing with notifications were implemented as native features.
Database Systems	Firebase Firestore and Supabase were employed as databases to store and retrieve data. The database schema was tailored to store and manage users, events, registrations, participation, and notifications. Real-time data sync (Firebase Firestore) combined with Supabase (additional relational database operations).

Human Computer Interaction (HCI)	Sphere's user interface is focused on ease-of-use, accessibility and user experience. Consistent interaction patterns, simple layout and informative messages were implemented throughout each module. The interface design strives for least number of interactions for accessing key features.
Computer Networks	Event Sphere relies on real-time interaction between the mobile application and the back-end by communicating using Firebase real- time listeners and REST APIs. Event Sphere employs HTTP requests to keep event data in sync, send push notifications and utilise Google Map services for location-based event management.
Object-Oriented Programming (OOP)	The app is designed using OOP concepts. Key concepts like User, Event, Registration, and Notification are represented as classes with encapsulated properties and behaviours. Inheritance, abstraction, and polymorphism are used in various modules.
Data Structures	Organisation of events, users and logs of attendance was handled by using efficient data structures. Lists, maps and queues are used for live data fetching, filtering events, and validating QR codes.

1.3 Project Background

A campus has numerous events that are essential for student, academic and community development. Yet, universities, including COMSATS University Islamabad, Sahiwal Campus, like most others, depend on a mix of manual and informal methods for managing events. Notices, WhatsApp groups and Facebook pages are used to announce events, while broadcast emails are sent to students. Students generally register for events through Google Forms or registration sheets and are marked present manually. This system is plagued by significant shortcomings: students receive notice in a piecemeal fashion, registration data is dispersed in multiple media and there is no convenient and central platform for managing attendance and tracking participation in events. Lack of a dedicated technology-based event management

system means that potential opportunities for student involvement are wasted, event planning and resource allocation becomes cumbersome, and there is no way of measuring engagement for future event planning.

Event Sphere is designed to address these issues. The app offers a role-based and unified platform for the entire event lifecycle to be managed securely and digitally. Event Sphere brings together all of the event-life-cycle activities in a single mobile application backed up by cloud data, removing the inefficiencies inherent in the manual approach.

1.4 Literature Review

Over the last decade, there has been significant advancement in research in the field of digital event management and university information systems using mobile technologies in particular. Research has consistently identified the necessity for integrated, technology-based systems that have the potential to take over manual processes in universities.

Several research has focused on the use of mobile apps for event management in universities. Studies show that when information about institutional activities is delivered to through mobile platforms students use, they are more likely to participate. Mobile technology enables it CLOSES the communication gap between university staff and students, especially in institutions with a large number of students, where it is difficult to communicate information through other channels (e.g. notice boards, email) in a timely and effective way.

The use of QR codes to check attendance in both academic and corporate settings has been widely researched. QR code-based attendance systems have been proven to minimise human error, prevent "buddy punching" and greatly save time on attendance recording. The use of backend-as-a-service (BaaS) solutions such as Firebase and Supabase has also been widely embraced in software projects for academic purposes owing to their scalability, real-time data processing capabilities and plug-and-play integration with mobile app development platforms. While the existing research supports the use of web-based event management systems, current research shows that a majority of universities are yet to adopt an integrated event management system. While there are many third-party solutions like Eventbrite and Meetup, these are not built for use within a university and lack the support for role- based access control of the university hierarchical structure. This study points to an opportunity that Event Sphere fills.

1.5 Analysis from Literature Review

Information from the literature review contributed to the design of Event Sphere and its functionality. The literature review brings in some significant findings that have implications for the project. First, it is widely accepted that mobile apps are an effective means of engaging students. This supports our choice to create Event Sphere as a mobile-first app with Flutter, meaning students, staff and administrators can easily use their mobile phone to access the system. Second, the reported success of QR-systems justified the inclusion of a QR- module. Third, the literature on the cloud-based backend supported the decision to use Firebase and Supabase. Lastly, the need for a complete integrated solution for university event management confirmed the novelty of Event Sphere.

1.6 Methodology and Software Lifecycle for This Project

Event Sphere was developed in accordance with the Agile Software Development Lifecycle (SDLC), chosen for its adaptability, iterative approach and compatibility with multiple modules that need to work seamlessly together, and a highly fluid requirements development. The software was developed by breaking the development into iterative sprints based on functional modules of the requirement. The details of the phases in developing Event Sphere are discussed in the following subsections.



Figure 1.1 Agile Model Diagram

Requirement Gathering:

The requirement gathering phase was the initial phase of the Event Sphere development. The team met with the project supervisor, co-supervisor and representative stakeholders (students and faculty) as part of a series of meetings to determine the system's primary requirements. The current system (manual event management process) was critically analysed to understand the deficiencies and determine the expectations for the new system.

The results of this phase were documented in the Software Requirements Specification (SRS) which will capture the functional and non-functional requirements of the system, use case descriptions, actor descriptions and system constraints. Use case diagrams were drawn to depict the interactions between actors and use cases. The requirement gathering phase provided a solid foundation that the rest of the design and development phases were based upon.

System Design:

Once the requirement phase was complete, the system design phase developed a detailed design or blueprint of Event Sphere. The system architecture was based on cloud-based client-server architecture with the mobile frontend implemented with Flutter and the server side implemented with Firebase along with Supabase.

This phase defined a three-layer architecture consisting of the Presentation Layer (all the Flutter-based UI components), the Application Layer (business logic and interactions with various services) and the Data Layer (all the interactions with Firestore, Supabase and Firebase Storage). A schema design was developed to specify the structure of all key entities namely: User, Event, Registration, Attendance, Notification, Expense and Society. Role- Based Access Control (RBAC) rules were defined to control access for Admin, Faculty and Students. Software Design Description (SDD) was used to document all design choices.

Model Development

During the model development phase, all UML and data flow diagrams - the design models of the Event Sphere system - were developed. These models visually explain the system's behaviour from the user's perspective and how its components communicate with each other. This phase resulted in the design of the following models: an Activity Diagram that represents the entire event management sequence of operations, from user login to certificate generation after the event; a Sequence Diagram that illustrates the sequence of operations in the approval process for events between the Faculty, Event Service, Firestore and the Notification Service; a

Class Diagram that represents all core entities of the system and their object-oriented relationships; a Data Flow Diagram Level 0 that presents an overview of the system in its environment; and a Data Flow Diagram Level 1 that breaks down the Event Management process into subprocesses. The set of these models provided a roadmap for the implementation phase of the project.

Implementation:

The implementation phase was the key development phase in the Event Sphere project. The development strictly followed the Agile methodology - a step by step approach to implementing the system in nine planned increments, each focused on building a particular system module. The initial increments set up the core functionality such as user authentication, role-based browsing of the app and simple event exploration. The following increments built on this foundation to incorporate event creation and approval processes, student registration and event attendance, QR code generation and scanning, sending push notifications, the application admin interface, analytics report generation, and a web landing page.

The mobile application was built using Flutter framework (written in Dart) using object-oriented programming principles [1]. Firebase Authentication was used to provide secure user login and roles determination. Firebase [2] Firestore was the main database used in real time while Firebase Cloud Messaging (FCM) was used for notifications. Firebase Storage was used to store the media files such as event poster and certificates. Google Maps API was also used to provide venue details of events.

Testing and Validation:

Testing was also done in parallel with implementation, with specific testing sessions at the end of the development of each increment. This is in line with the principles of Agile to catch defects early on to prevent them from spreading throughout the system.

Unit testing was conducted on individual functions and UI elements to ensure they functioned as expected. Integration testing was conducted to ensure modules were integrated properly by passing data between the Flutter front end and Firebase back end. System testing verified the entire system with the functional and non-functional requirements specified in the SRS. User Acceptance Testing (UAT) was performed with a group of representative students and professors to ensure ease-of-use of the system. Concurrency and load testing were conducted to check if the system responded in a timely manner to a specified throughput and load.

Deployment:

In the deployment phase, the Event Sphere application was made accessible to the target audience. The Android app was built into an APK and published for download via the promotional web landing page, created as part of this project. The page includes a description of the app's features, screenshots, and a link to download the APK file.

The Firebase and Supabase DataBase as a Service (DBaaS) services were set up to run on their respective hosting platforms and all of the database rules, authentication settings, storage rules and Firebase notification service credentials were verified to be in place for the production environment. Firebase Security Rules were finalised to control access to data, ensuring role-based access control at the database level, with no unauthorised read/write access to any collection of data.

Maintenance:

The maintenance stage is the cycle of activities needed to maintain the stability, security and functionality of the Event Sphere platform after it has been deployed. Maintenance tasks include analysing system logs using the Firebase console and the Event Sphere administration logging module, fixing identified bugs uncovered in use, maintaining security (patches) and dependency (updates) management, and improving the efficiency of database queries as the amount of data grows.

The highly modular design of Event Sphere, source code under version control, and naming conventions for various objects, ensure maintenance can be conducted in a modular approach without impacting the overall system. Event Sphere is planned to be extended with additional features such as AI event recommendations, multi-campus support, offline caching and support for the iOS mobile platform in future maintenance iterations, which could be implemented as incremental development work items without impact on existing functionality.

Rationale behind Selected Methodology:

Event Sphere has opted for an Agile approach for several reasons. The system is made up of various modules that have individual specifications and complexities; therefore, a waterfall methodology could not be used. The Agile methodology with its iterative nature allowed each module to be built and tested independently, minimizing the possibility of integration issues. The approach also allowed for inevitable evolving requirements, which arose as work was being done

and input from the supervisors was included. Given the team involved in the project was three people, Agile's focus on communication and collaboration with continuous delivery was well suited to the project team.

Chapter 2

Problem Definition

2 Problem Definition

This chapter offers a full definition of the problem to be addressed by Event Sphere. It describes the problems that Universities face when dealing with academic events through traditional manual processes; examines the deliverables and development needs in the proposed system; and provides a thorough description of the current system and its problems.

2.1 Problem Statement

Universities are vibrant campuses, hosting a variety of events throughout the year. While these events are crucial for the development and engagement of students and the university, planning, communication, and the participation in university events at COMSATS University Islamabad, Sahiwal Campus is largely still manual, unintegrated and poorly managed.

Core Problem Statement:

There is no digital, integrated system for event management for university events that links event creation, administrative approval, student event registration and attendance, expenses and real-time communication and provides access to all this in a role-based, integrated system. This leads to event data being spread over multiple channels, inefficient and inaccurate student registration, manual attendance tracking, lack of expense tracking and a system that cannot be used for data analysis to oversee and manage events. This fundamental issue impacts three key stakeholder segments, with their own unique, yet related, challenges. Students experience poor communication and nonstandardised registrations. Academics suffer from inappropriate approval processes, tedious attendance management and lack of analytics to measure student's participation. Administrators do not have a central tool to monitor, have no unified data and cannot enforce institutional regulations through a formal process.

All of the aforementioned challenges are addressed by Event Sphere, which delivers a scalability, cloud-based, and role-based mobile application that digitises the event management workflow from the event idea through to approval and registration, to student attendance (through QR codes), expense management, instant event updates and post - event reporting.

Stakeholder-Specific Challenges:

The challenges created by the absence of a centralized event management system impact all three stakeholder groups in distinct but interrelated ways. Students face fragmented event communication through informal channels (WhatsApp, notice boards), lack of a searchable

event repository, unreliable registration through Google Forms or paper sheets that cannot prevent duplicates, time-consuming manual attendance via printed lists, and no automated event reminders. Faculty experience disjointed event announcements across multiple channels, the absence of a formal online event submission and approval process, no real-time access to participant lists, unstandardized expense management, and no systematic event feedback mechanism. Administrators lack a centralized event approval system risking inappropriate event listings, have no aggregated event analytics or participation data, manage user accounts manually increasing the risk of errors, and have no moderation tools for outdated or cancelled events.

Table 2.1: Stakeholder Specific Challenges

No.	Deliverable	Description
D01	Mobile Application (Android)	A complete Android Flutter (Dart) mobile application that provides role- based (Student, Vice President, President, Admin, Super Admin) access with event management features.
D02	User Authentication System	A robust Firebase authentication module for user registration, login, password recovery and email verification with Role-based Access Control (RBAC).
D03	Event Management Module	A full event management module to enable event creations by faculty and event submissions to admin for approval, reject or update.
D04	QR Code Attendance System	An automated QR code generator and scanner module creating unique QR codes for each event, verifying student entries, updating Firestore with attendance data, and certifying events for attendance.
D05	Real-Time Notification System	A Firebase Cloud Messaging (FCM) integration to provide real-time notifications for event approvals, event registration confirmations, reminders and calendar changes, as well as notifications from the administrators.

D06	Admin Dashboard and Reports	An admin portal for user and event management, attendance tracking, downloadable reports and ensuring the integrity of the system.
D07	Expense Management Module	A functionality for faculty to claim expenses incurred in an event, upload receipts for review and processing; for admins to approve or deny expense claims with notifications of their actions.
D08	Promotional Web Landing Page	A public webpage that showcases Event Sphere's features, screenshots, with a link to download the Android APK.
D09	SRS Document	A formal Software Requirements Specification documenting functional and non-functional requirements, use case diagram, actor specifications, constraints and others.
D10	SDD Document	A formal Software Design Description, including system architecture, UML diagrams, data dictionary and algorithm descriptions.
D11	Final Project Report	An academic report describing all aspects of the project such as its introduction, problem statement, requirements, design, implementation, testing and conclusions.

2.2 Development Requirements

Alongside the deliverables, the Event Sphere project also requires a number of technical, functional and non- functional development requirements to be met. These outline the restrictions, rules and needs of the system during the development process.

Technical Requirements:

- Primary mobile development must be done using Flutter (Dart) framework to run on Android

devices without performance issues.

- The system must implement Firebase as the primary backend service for Authentication, Cloud Firestore Database, Firebase Cloud Messaging (FCM) and Firebase Storage.
- Supabase must be used as a secondary (optional) backend to provide additional relational databases [3].
- The application must have Role-based access control (RBAC) that must implement, at both client and server side, strict access control for Admin, Faculty and Student roles. It must leverage Google Maps API [4] for event location, mapping and venue management.
- The QR code system must allow automatic generation of QR codes for each event and seamlessly include a QR code scanner to check attendance.

.

2.2.1 Functional Requirements

- The system shall provide full user lifecycle management in terms of registration, login, user profile update and user role management for all user roles.
- Academics must be able to add, edit and submit events which are reviewed and approved by administrators.
- Event podcasts must be approved or rejected by admins with notes sent to the event organisers via push notifications.
- Students have access to browse, search and filter, register for and bookmark events through the mobile app.
- The expenses module should provide an option for faculty to upload expenses (along with receipts) and for administrators to approve/reject expenses.
- Admin dashboard must include event analytics, user management and event reports download.

2.2.2 Non-Functional Requirements:

Performance: Event list loading time should be under two seconds, with a user load of at least 500 users.

- Scalability: The server architecture should allow for horizontal scaling for a growing number of users, events, and records.
- Security: Data collected in the system should be transmitted using SSL/TLS, and Firebase Security Rules should be used to enforce role-based database access control.
- Usability: The user interface should be simple and user-friendly, not requiring technical

expertise for any role.

- **Reliability:** The system should have at least 99.9% uptime during term time.
- **Maintainability:** The system must be structured according to OOP principles and must have well separated modules for ease of modification.

2.3 Current System

Providing a background on the current system is needed to better understand the level of improvement brought by Event Sphere. The current system for university events is a mixture of manual processes, ad hoc communications and digital systems.

Comparison: Current System vs. Event Sphere

The following table shows a head-to-head comparison of the existing manual system and the new Event Sphere system in terms of operational aspects.

Table 2.2: Current System vs. Event Sphere

Aspect	Current Manual System	Event Sphere Solution
Event Announcement	Physical notice boards, WhatsApp groups, Facebook pages, email broadcasts	Centralized in-app event listing with real-time Firestore updates and FCM push notifications
Event Registration	Google Forms or manual paper sign-up sheets	One-tap digital registration with automatic participant list generation and capacity enforcement
Attendance Recording	Manual sign-in sheets managed by event organizers	QR code-based attendance scanning with instant Firestore recording and duplicate check prevention
Event Approval	Informal verbal or email-based approvals	Structured digital approval workflow with admin review

		queue, status tracking, and feedback notes
Notifications and Updates	WhatsApp messages, phone calls, or email broadcasts	Automated Firebase Cloud Messaging (FCM) push notifications for all system events
Participation Tracking	No centralized tracking; records in separate files	Real-time participation analytics visible to faculty and administrators through the dashboard
Event Discovery	Students depend on chance encounters with notice boards	Searchable and filterable event listing with category, department, and date filters
Expense Management	No formal expense tracking mechanism	Dedicated expense submission module with receipt uploads and admin approval workflow
Reporting and Analytics	No formal reporting mechanism available	Admin dashboard with downloadable participation reports and visual engagement statistics

The comparison we present in Table 2.2 shows how Event Sphere overcomes each of the challenges of the existing system, in a specific and specific way. Event Sphere has indeed transformed the current manual and decentralized event management process with a role-based, automated and digital platform.

Chapter 3

Requirement Analysis

3 Requirement Analysis

This chapter provides a full analysis of the Event Sphere system's requirements sourced from the Software Requirements Specification (SRS). This includes the Use Case Diagram, Use Case Descriptions, Functional Requirements (grouped by application feature) and Non-Functional Requirements (grouped by quality attribute). All requirements can be traced back to a specific stakeholder need and the problem statement defined in Chapter 2.

3.1 Use Case Diagrams

In this Use case diagram, we show the behavior of our system. This enables you to visualize the different types of roles in a system and how those roles interact with the system. It outlines the various actions and functionalities that the system provides to its users.

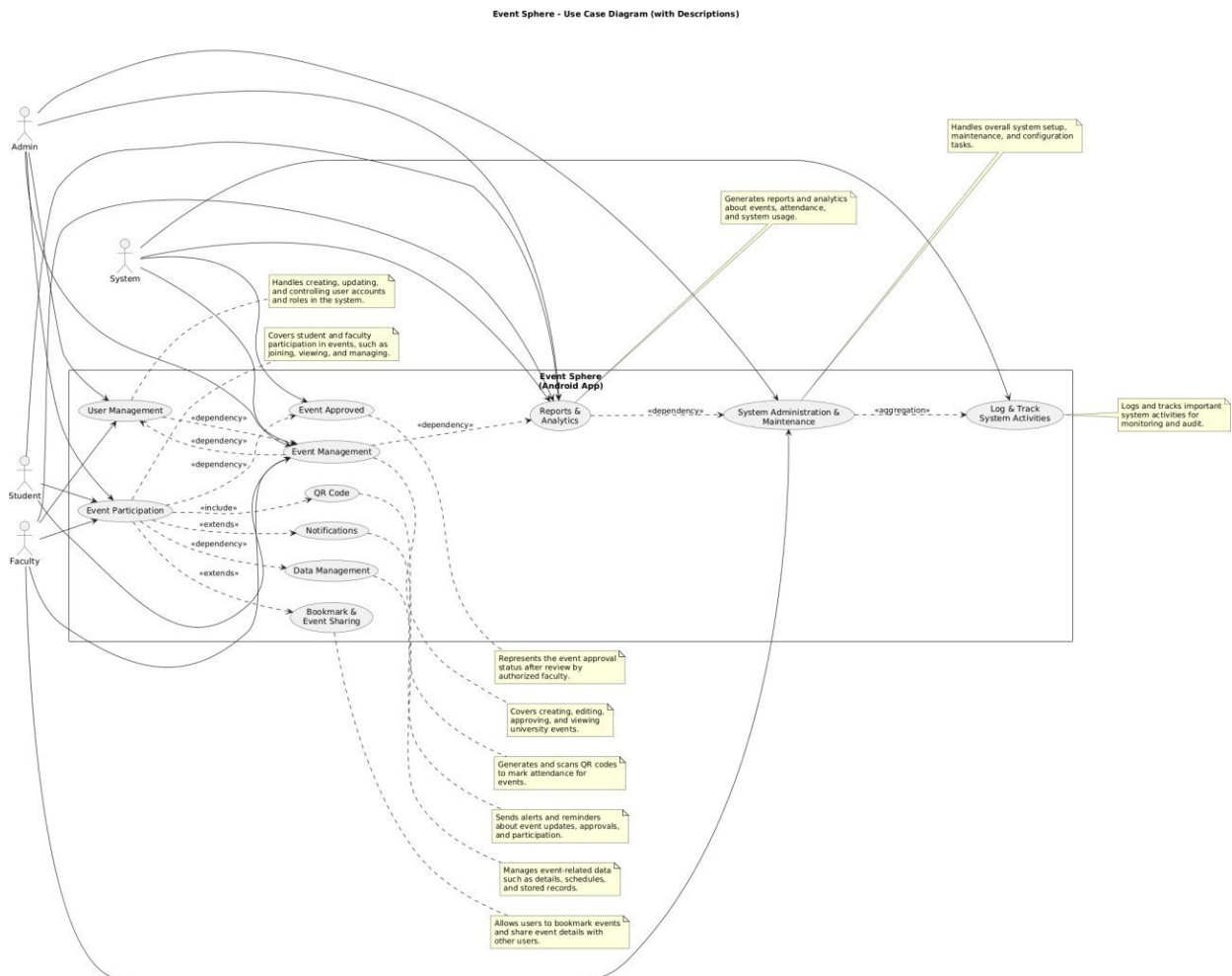


Figure 3.1 Use Case Diagram

The Event Sphere Use Case Diagram is a top-level view of interactions between key stakeholders and functional components of the system. This diagram defines the full set of system functionality from the point of view of each stakeholder role and defines the system boundaries of the system being built. Figure 3.1 of the SRS document refers to the diagram.

3.1.1 Actors

There are four key actors in Event Sphere, with different roles and interaction with the system:

Admin

Event Sphere's administrator has the highest level of authority. The Admin is in charge of user management, reviewing and approving or denying event submissions from Faculty, maintenance and configuration of the system, moderating the content of the system, and reviewing all reports, including the statistical reports of attendance. The Admin is involved with all five main use cases.

Faculty

Faculty are event organisers. They can propose and submit events, manage events they have organized, view participant registrations for events they have proposed and organized, submit expenses for events they have organized and access reports and analytics for their events. The Event Management, Event Participation, Reports and Analytics, and Expense Management use cases are the main interactions faculty have with the system.

Student

The target audience of Event Sphere is students. They attend and find approved events, sign up to attend events with one click, get push notifications for event changes and reminders, mark events they will attend, and confirm their attendance on the day of the event using a QR code. Students engage with the Event Participation and Notifications use cases.

System

The System actor is the automated system processes that run in the background. The system is responsible for sending Firebase Cloud Messaging (FCM) [5] notifications, creating unique QR codes for events that are approved, checking whether a participant is attending an event, processing status requests for events, backing up data, and maintaining a log of all system activities. It facilitates and supports the interactions of all other actors in the event process.

3.1.2 Primary Use Cases:

The Event Sphere system is made up of five primary use cases, each corresponding to a key functional component of the application:

- UC-1 User Management: Represents all of the user account lifecycle functions (create, update, delete, assign role) by the Admin with the support of System.
- UC-2 Event Management: Manages the entire event life cycle from Faculty creation and submission to approval by the Admin and publication to students, involving Admin, Faculty and System actors.
- UC-3 Event Participation: Involves student event registration, digital one-tap sign-ups, QR code based attendance check and certificate eligibility management by Student, Faculty and System actors.
- UC-4 Reports and Analytics: Covers creation, viewing and exporting of event participation, attendance, and engagement reports, and analytics, involving Admin and Faculty, and System.
- UC-5 System Administration and Maintenance: Covers system configuration settings, data backup, log activity creation, system issue resolution and system "well-being" management by Admin, with System support.

3.1.3 Use Case Relationships

- Event Management use case includes optional Bookmark and Event Sharing.
- Event Participation has an optional QR Code component for check-in.
- Event Participation includes the Notifications module for confirmations, reminders and notifications.
- The Data Management module has dependency with the Event Management, Event Participation and Reports and Analytics.
- System Administration and Maintenance aggregates with the Log and Track System Activities module.

3.2 Detailed Use Case Descriptions

This section provides the full detailed use case descriptions of all the five use cases, documented using a template that includes identifier, actors, description, trigger, preconditions, postconditions, normal flow, alternative flows, exceptions and business rules.

3.2.1 Use Case Summary

The following summarizes the five primary use cases of Event Sphere with their key actors, workflows and business rules:

UC-1 User Management: The Admin manages all user accounts (create, update, delete) for Faculty and Student users. The system validates inputs, enforces email uniqueness, updates the database, and logs all operations. Only Admin users may execute user management operations and every account must have a unique ID with a verified institutional email.

UC-2 Event Management: Faculty create and submit event proposals with full details. Submissions enter an Admin approval queue for review. The system manages validation, database storage, and automated notifications. Only approved events are visible to students, and rejected events must include mandatory Admin feedback. If modifications are needed, the event is returned to Faculty for resubmission.

UC-3 Event Participation: Students browse approved events and register digitally through a one-tap process. On event day, students scan a unique QR code to confirm attendance. The system validates the scan (event ID match, registration check, duplicate prevention), records attendance, and marks certificate eligibility. Only registered students may scan the event QR code, and each QR code is unique per event and single-use per student.

UC-4 Reports and Analytics: Admin and Faculty generate detailed reports on event activity, participation rates, attendance statistics, and departmental engagement. The system processes data into visual charts and exportable PDF/Excel formats. Faculty can only access analytics for events they created.

UC-5 System Administration and Maintenance: The Admin maintains platform stability through system configuration updates, data backups, activity log reviews, user role management, and issue resolution. The system automates monitoring and scheduled backups. All system activities must be logged with a minimum 6-month retention period.

3.3 Functional Requirements

The functional requirements specify the functionality that the Event Sphere system needs to support the interactions with its actors. The requirements are grouped below by feature, each feature is presented in a sub-section and the requirements for that feature are presented in a table. This allows us to link system features to their requirements.

3.3.1 User Registration and Login

Table 3.1: Functional Requirements — User Registration and Login

FR ID	Priority	Actor	Requirement Description
-------	----------	-------	-------------------------

FR-1.1	High	Admin	The administrator shall be able to create, update, and delete user accounts for Faculty and Student users within the system.
FR-1.2	High	System	The system shall validate all user input fields including email format, password strength, and role assignment before storing in Firestore.
FR-1.3	Medium	System	Password reset links must expire within 15 minutes of issuance to prevent unauthorized access through stale tokens.

3.3.2 Event Creation and Management

Table 3.2: Functional Requirements — Event Creation and Management

FR ID	Priority	Actor	Requirement Description
FR-2.1	High	Faculty	Faculty shall be able to create new event proposals containing title, description, date, time, venue, category, and society association.
FR-2.2	High	Admin	The administrator shall approve or reject event proposals submitted by Faculty, with mandatory feedback notes for rejected submissions.
FR-2.3	Medium	System	The system shall automatically send push notifications to Faculty upon approval or rejection of their submitted event proposal.

3.3.3 Event Registration and Participation

Table 3.3: Functional Requirements — Event Registration and Participation

FR ID	Priority	Actor	Requirement Description
FR-3.1	High	Student	Students shall be able to view all approved events and register through a one- tap digital registration process with automatic seat management.

FR-3.3	Medium	System	The system shall send automated confirmation notifications upon registration and reminder notifications 24 hours before the scheduled event.
---------------	---------------	--------	--

3.3.4 QR Code Scanning and Attendance

Table 3.4: Functional Requirements — QR Code Scanning and Attendance

FR ID	Priority	Actor	Requirement Description
FR-3.2	High	System	The system shall generate unique QR codes for each approved event and provide a built-in scanner for student attendance verification on the event day.

3.3.5 Notifications

Table 3.5: Functional Requirements — Notifications

FR ID	Priority	Actor	Requirement Description
FR-2.3	Medium	System	The system shall deliver event status notifications (approval, rejection, updates) to Faculty and event reminder notifications to registered students.
FR-3.3	Medium	System	Registration confirmation and 24-hour reminder notifications shall be triggered automatically without any manual intervention.

3.3.6 Reports and Analytics

Table 3.6: Functional Requirements — Reports and

FR ID	Priority	Actor	Requirement Description
FR-4.1	High	Admin	The administrator shall generate comprehensive event participation reports including event name, participation rate, attendance count, and feedback summary.

FR-4.2	Medium	Faculty	Faculty shall access attendance analytics and engagement trends for the events they have created and managed.
FR-4.3	Medium	System	The system shall generate visual charts and statistical summaries with export functionality supporting PDF and Excel formats.

3.3.7 Expense (Cost) Management

Table 3.7: Functional Requirements — Expense (Cost)

FR ID	Priority	Actor	Requirement Description
FR-5.1	High	Admin	The administrator shall manage system configurations, apply updates, and oversee expense approval workflows without interrupting active user sessions.
FR-5.2	High	System	The system shall log and maintain records of all administrative activities and expense submissions with a minimum retention of six months.
FR-5.3	High	System	The system shall perform automated daily backups of all event, user, and expense data at midnight server time with integrity verification.

3.3.8 System Administration and Maintenance

Table 3.8: Functional Requirements — System Administration and Maintenance

FR ID	Priority	Actor	Requirement Description
FR-5.1	High	Admin	The administrator shall manage user roles, system configurations, and maintenance operations including data backups and system health checks.

FR-5.2	High	System	The system shall log all administrative actions and track system activities for auditing and accountability.
---------------	-------------	--------	--

3.4 Non-Functional Requirements

Non-functional requirements specify the quality characteristics and constraints that apply to the way Event Sphere operates. They are grouped below into quality categories, and are each listed in a separate sub-section, detailing the performance criteria that Event Sphere must satisfy.

3.4.1 Usability Requirements

Table 3.9: Usability Requirements

<i>NFR ID</i>	<i>Requirement</i>
<i>NFR-1.1</i>	<i>Key features such as event registration and report generation shall be accessible within three clicks from the main dashboard.</i>
<i>NFR-1.2</i>	<i>The user interface shall maintain consistent layout, color themes, and navigation controls across all modules for all user roles.</i>
<i>NFR-1.3</i>	<i>The system shall display clear validation and error messages to guide users through form submissions and corrections.</i>
<i>NFR-1.4</i>	<i>The application shall require no specialized technical training and shall be accessible to users with basic digital literacy.</i>

3.4.2 Performance Requirements

Table 3.10: Performance Requirements

<i>NFR ID</i>	<i>Requirement</i>
<i>NFR-2.1</i>	<i>The system shall respond to all user interactions within two seconds under normal network conditions.</i>

<i>NFR-2.2</i>	<i>Event creation, editing, and deletion operations shall complete within five seconds of initiation.</i>
<i>NFR-2.3</i>	<i>The system shall support simultaneous access by at least 500 active users without performance degradation.</i>
<i>NFR-2.4</i>	<i>QR-based attendance verification shall be processed and confirmed within three seconds of a valid scan.</i>
<i>NFR-2.5</i>	<i>Report generation for datasets of up to 10,000 records shall complete within ten seconds.</i>

3.4.3 Scalability Requirements

Table 3.11: Scalability Requirements

<i>NFR ID</i>	<i>Requirement</i>
<i>NFR-3.1</i>	<i>The system architecture shall support both horizontal and vertical scaling to accommodate future growth in users and events.</i>
<i>NFR-3.2</i>	<i>The system shall support a minimum of 10,000 registered users and 1,000 concurrent sessions with minimal latency.</i>
<i>NFR-3.3</i>	<i>The database schema shall accommodate addition of new data entities and modules without redesigning the core structure.</i>

3.4.4 Availability Requirements

Table 3.12: Availability Requirements

<i>NFR ID</i>	<i>Requirement</i>
<i>NFR-4.1</i>	<i>The system shall maintain 99.9% uptime excluding planned and communicated maintenance windows.</i>
<i>NFR-4.2</i>	<i>In the event of server failure, the system shall recover from the most recent verified backup within 15 minutes.</i>
<i>NFR-4.3</i>	<i>The system shall implement automated restart and failover mechanisms to ensure continuous operation.</i>

3.4.5 Compatibility Requirements

Table 3.13: Compatibility Requirements

NFR ID	Requirement
NFR-5.1	The mobile application shall be compatible with Android devices running Android 8.0 or higher.
NFR-5.2	The web-based landing page shall be compatible with all major modern browsers including Chrome, Firefox, Edge, and Safari.
NFR-5.3	The application shall support integration with institutional email systems for notifications and account verifications.

3.4.6 Maintainability Requirements

Table 3.14: Maintainability Requirements

NFR ID	Requirement
NFR-6.1	The system shall be modular, allowing independent maintenance of features without affecting other modules.
NFR-6.2	The System Administration module shall record comprehensive logs of all activities, configurations, and errors.
NFR-6.3	All source code shall follow documented naming conventions and be managed under Git version control throughout development.

The non-functional requirements specified above collectively define the operational quality baseline for Event Sphere. Together they ensure the system is accessible, fast, scalable, always available, broadly compatible, and easy to maintain and extend over time.

Chapter 4

Design and Architecture

4 Design and Architecture

This chapter outlines the full design and architecture specification of the Event Sphere system as outlined in the Software Design Description (SDD). It outlines the three-layer system architecture, data model representation and entity relationships, workflow model represented in the activity diagram, and formal design models such as sequence diagram, class diagram and data flow diagrams. Collectively, these components offer a holistic view of the structure and its behaviour that underpins the design of the system.

4.1 System Architecture

Event Sphere is based on the three-layer client-server architecture with cloud services. This ensures real-time event notifications, secure delivery of data, a scalable backend service and seamless communication between all user types. The three layers of the Event Sphere architecture are the Presentation, Application and Data Layer, with clearly demarcated responsibilities and boundaries.

4.1.1 Presentation Layer

The Presentation Layer is the mobile application interface of Event Sphere and constitutes all of the user interface elements that the end user interacts with on their Android phone. It is implemented entirely using Flutter and Dart, a framework for building user interfaces entirely from widgets that are rendered with high fidelity and responsiveness.

The Presentation Layer is structured into five different role-based interfaces, rendered at runtime depending on the role of the logged-in user:

Student Interface

The Student interface allows access to the event discovery and browsing screen, the event registration module with one-tap registration, the QR code scanner for attendance logging on the day of the event, the bookmarks list for storing events of interest, the notifications module to view event registrations and reminders, and the user profile screen to update user details and view participation history.

Faculty Interface

The Faculty interface offers the event creation and management form, the pending events list indicating the current status of submissions, the participant list and attendance screen for

approved events, the expense management module with receipt upload feature, and the analytics dashboard displaying participation trends and attendance data for events created by Faculty members.

Admin Interface

The Admin interface offers access to the user management panel for adding, editing and deleting Faculty and Student users, the event approval queue for reviewing and approving pending events, the system-wide analytics and participation reports dashboard, the expense approval process, the content moderation tools for deleting invalid or expired events, and the system administration screen for configuration and activity log.

Vice President Interface

The Vice President interface provides event management capabilities within their assigned society, including event creation and submission for approval, participant list viewing, and society member coordination. Vice Presidents can manage events associated with their society and view analytics for events they have organized.

President Interface

The President interface extends the Vice President capabilities with additional society management features including member management (adding/removing members), society profile editing, and the ability to create announcements for society members. Presidents serve as the primary point of contact between the society and the administration.

All user interface screens in the Presentation Layer include form validation prior to transmitting data to the backend server, real-time updates triggered by Firebase Firestore [6] database listeners, and a consistent user experience with common themes such as consistent use of color, navigation, and error messages across all modules.

4.1.2 Application Layer

The Application Layer is the business and service integration layer of Event Sphere. This layer is located between the Presentation Layer and the Data Layer, and is responsible for handling user interactions, applying business rules, orchestrating service operations and managing the application state. The Application Layer is implemented in the Flutter application as service and state management classes.

Authentication Service

The Authentication Service handles all user authentication via Firebase Authentication. This includes user registration, password verification, session tokens, password resets and email verification. Most importantly, it is responsible for Role-Based Access Control (RBAC) via reading the user role from Firestore after login and providing the appropriate role-based view to the application. Only authenticated users with a role can go beyond the login screen.

Event Service

The Event Service handles the entire event lifecycle. It manages event creation (including input validation and Firestore write) by Faculty, event retrieval (filtered by status=Approved) for student browsing, event status update (approval/rejection with comments) by Admin, and real-time event data synchronization to all devices via Firestore listeners. Furthermore, the Event Service integrates with the Notification Service to send automatic push notifications at various events lifecycle stages.

Registration and Attendance Service

The Registration Service handles student registration for events using Firestore transactions to check event seat availability and reduce it in an atomic way, avoiding overbookings even if multiple clients try to register at the same time. It records student sign-ups, ensures that students can only sign up for one event and sends confirmation emails. The Attendance Service generates QR codes [7] for each registered event and validates QR code scans, ensuring the student is registered, the event matches, and that the student is not checked in twice, then logs the attendance in Firestore.

Notification Service

The Notification Service uses Firebase Cloud Messaging (FCM) to provide instant notifications to all users. It is automatically triggered by other services on some important events such as event creation, approval, rejection, student enrolment, 24-hour event reminder, schedule change and system announcement. Notifications are also saved in the Firestore Notifications collection to be displayed in the user's notification center in the app.

Expense Service

The Expense Service handles event expenses. Faculty create expense records, including the value, category, description, and upload the receipt photo to Firebase Storage. The service records expense information in Firestore and sends a notification for Admin approval. The service receives the Admin's approval or rejection and updates the expense status in Firestore and sends the Faculty member a notification of the decision.

4.1.3 Data Layer

The Data Layer is the component of Event Sphere that handles all persistent data storage, access and management. It consists of three different back-end services that offer real-time synchronization, relational data storage and management, media storage and secure access control.

NoSQL Database (Firebase Firestore)

The main real-time database of Event Sphere is Firebase Cloud Firestore. It maintains the data of all the major components of the application in a NoSQL document-collection format in the following collections: Users (storing Student, Vice President, President, Admin, and Super Admin information), Events (storing all events with status, organizer, and approval details), Registrations (storing student sign-up details), Attendance (storing QR code-verified student attendance records), Notifications (storing system-generated notifications), Expenses (storing faculty expense claims), and Societies (storing university societies information). Firestore's real-time listeners automatically update the database changes in real-time to all connected clients without needing to refresh.

Supabase (Supplementary Backend)

Supabase is used as an extended backend system to address additional relational data storage needs that extend beyond Firestore's NoSQL capabilities. It offers relational storage and can be used as a second storage layer to support data operations that require a relational schema (such as complex queries and joins between multiple entities).

Firebase Storage (Media Storage)

Firebase Storage is used for storing all binary files in Event Sphere. It handles the storage of event poster images uploaded by Faculty, the storage of images and PDFs for expense receipts, certificates of attendance for students who have registered for events, and any other media used

for promotion. All files are stored with secure tokens and their download URLs are stored in the Firestore document records for the application to access.

Firestore Security Rules

Firestore Security Rules are set for Firestore and Firebase Storage to implement role-based data access controls. These determine that Student users can only read approved events and their own event registration and attendance records, Faculty users can only write events they created, and that only Admin users can access all collections with full read and write rights. Data communication between the mobile client and Firebase services is secured with SSL/TLS.

4.2 Data Representation

This chapter presents a full description of the Event Sphere data design, specifying how the information is represented, stored and related in the system. The data in Event Sphere is stored in the Firebase Firestore database (a document-collection NoSQL database) and in Firebase Storage for all binary media files. The data structure is defined using the data dictionary, entity relationships and the conceptual Entity-Relationship (ER) model.

4.2.1 Data Dictionary

The data dictionary defines all primary data entities used within Event Sphere and specifies the fields, data types, and roles of each attribute. Table 4.1 presents the complete data dictionary for the system.

Table 4.1: Data Dictionary — Event Sphere

Entity	Fields
User	userId (PK), fullName, email, department, role (Admin/Faculty/Student), profileImageUrl, createdAt, updatedAt
Event	eventId (PK), title, description, venue, date, time, capacity, organizerId (FK→User), societyId (FK→Society), status (Pending/Approved/Rejected), approvalNotes, approvalDate, posterUrl
Registration	registrationId (PK), userId (FK→User), eventId (FK→Event), registrationDate, status (Registered/Cancelled)

Attendance	attendanceId (PK), userId (FK→User), eventId (FK→Event), checkInTime, verified (Boolean), certificateEligible (Boolean)
Notification	notificationId (PK), userId (FK→User), message, type (Approval/Reminder/Update/System), timestamp, seen (Boolean)
Expense	expenseId (PK), eventId (FK→Event), submittedBy (FK→User), amount, category, receiptUrl, status (Pending/Approved/Rejected)
Society	societyId (PK), name, description, presidentId (FK→User), memberCount
AI Log (Optional)	logId (PK), userId (FK→User), prompt, response, timestamp
DeviceToken	tokenId (PK), userId (FK→User), fcmToken, deviceInfo, createdAt

These entities that have been specified in Table 4.1 make up the full data model of Event Sphere. These entities are stored in the form of Firestore collections with their respective records being stored in the form of documents. The foreign key relationship between these collections can be established through the use of document references linking these documents across other collections.

In addition to the nine primary entities listed in Table 4.1, the Event Sphere system includes several supplementary entities that support extended functionality:

Table 4.1b: Supplementary Data Entities - Event Sphere

EventFeedback: feedbackId (PK), eventId (FK→Event), userId (FK→User), rating (Integer 1-5), comment (Text), createdAt. Stores star ratings and written reviews submitted by students after attending events.

EventWaitlist: waitlistId (PK), eventId (FK→Event), userId (FK→User), position (Integer), joinedAt, promotedAt. Manages a queue of students waiting for spots when events reach full capacity.

EventReactions: reactionId (PK), eventId (FK→Event), userId (FK→User), reactionType (like/dislike), createdAt. Enables students to express interest in events through like/dislike interactions.

EventComments: commentId (PK), eventId (FK→Event), userId (FK→User), content (Text), parentId (FK→EventComments, nullable), createdAt. Provides threaded discussion capabilities on event pages. EventPhotos: photoId (PK), eventId (FK→Event), userId (FK→User), photoUrl (Text), caption (Text), isApproved (Boolean), uploadedAt. Supports a photo gallery feature for capturing event memories.

EventResources: resourceId (PK), eventId (FK→Event), title (Text), fileUrl (Text), fileType (document/pdf/image/video/link), uploadedBy (FK→User), uploadedAt. Allows organizers to attach downloadable files and links to events.

EventCommittees: committeeId (PK), eventId (FK→Event), userId (FK→User), role (head/coordinator/volunteer), responsibilities (Text), joinedAt. Manages the organizing team assigned to each event.

UserPoints: pointsId (PK), userId (FK→User, unique), totalPoints (Integer), eventsAttended (Integer), badges (Text Array), createdAt, updatedAt. Supports the gamification system with points, badges, and a leaderboard. RoleRequests: requestId (PK), userId (FK→User), requestedRole (Text), reason (Text), status (pending/approved/rejected), reviewedBy (FK→User), reviewedAt, createdAt. Manages role upgrade requests submitted by users.

RoleChanges: changeId (PK), targetUser (FK→User), oldRole (Text), newRole (Text), changedBy (FK→User), changedAt. Provides an audit log for all role changes in the system.

4.2.2 Entity Relationships

The entity relationships specify how the primary data entities of Event Sphere are related and data exchange occurs among them. Knowing these relationships is important in order to formulate database queries as well as to establish the proper Firestore security rules. The following table provides a summary of the entity relationship specification.

Table 4.2: Entity Relationship Specification — Event Sphere

Relationship	Cardinality	Description
User → Event	One to Many	A single User (Faculty) can create many Events. Each Event has exactly one organizer.

User → Registration	One to Many	A single User (Student) can have many Registrations. Each Registration belongs to exactly one User.
User→ Registration	One to Many	le Event can have many Registrations. Each Registration is linked to exactly one Event.
User → Attendance	One to Many	A single User can have Attendance records across many Events. Each Attendance record belongs to one User.
Event → Attendance	One to Many	A single Event generates many Attendance records, one per verified student check-in.
Event→ Notification	One to Many	A single Event can trigger many Notifications (approval, reminder, update). Each Notification references one Event.
User → Notification	One to Many	A single User can receive many Notifications. Each Notification is addressed to one User.
Event → Expense	One to Many	le Event can have multiple Expense submissions by Faculty. Each Expense is linked to one Event.
Society → Event	One to Many	A single Society can organize many Events. Each Event references one Society.
User → Society	Many to Many	User can be members of multiple Societies. Each Society can have multiple members.

4.2.3 ER Diagram Overview

The conceptual Entity-Relationship (ER) model of Event Sphere has the Event entity as the central source of data for other entities. The most frequently referenced entity is the User entity, as a foreign key of Events (as event organizer), Registrations (as event participant), Attendance (as event attendee), Notifications (as event recipient) and Expense (as expense submitter). This prominence of User is an outcome of the system's role-based architecture where every major action is initiated by and can be traced back to a user.

Event connects to Registration, Attendance, Notification, and Expense, representing the entire event lifecycle: a Faculty User creates an Event, Students register using a Registration record, check in using an Attendance record, Notification records are automatically created at each step in the lifecycle, and Faculty submit Expense records for the event. The Society entity links to Event through a one-to-many relationship, reflecting that each university society can host numerous events each year.

The ER model is implemented in Firebase Firestore as a document-collection database where each entity is a collection, a relationship to another entity is represented as a reference to a document in that entity, and Firestore Security Rules control access to collections and documents based on a user's role.

Activity Diagram

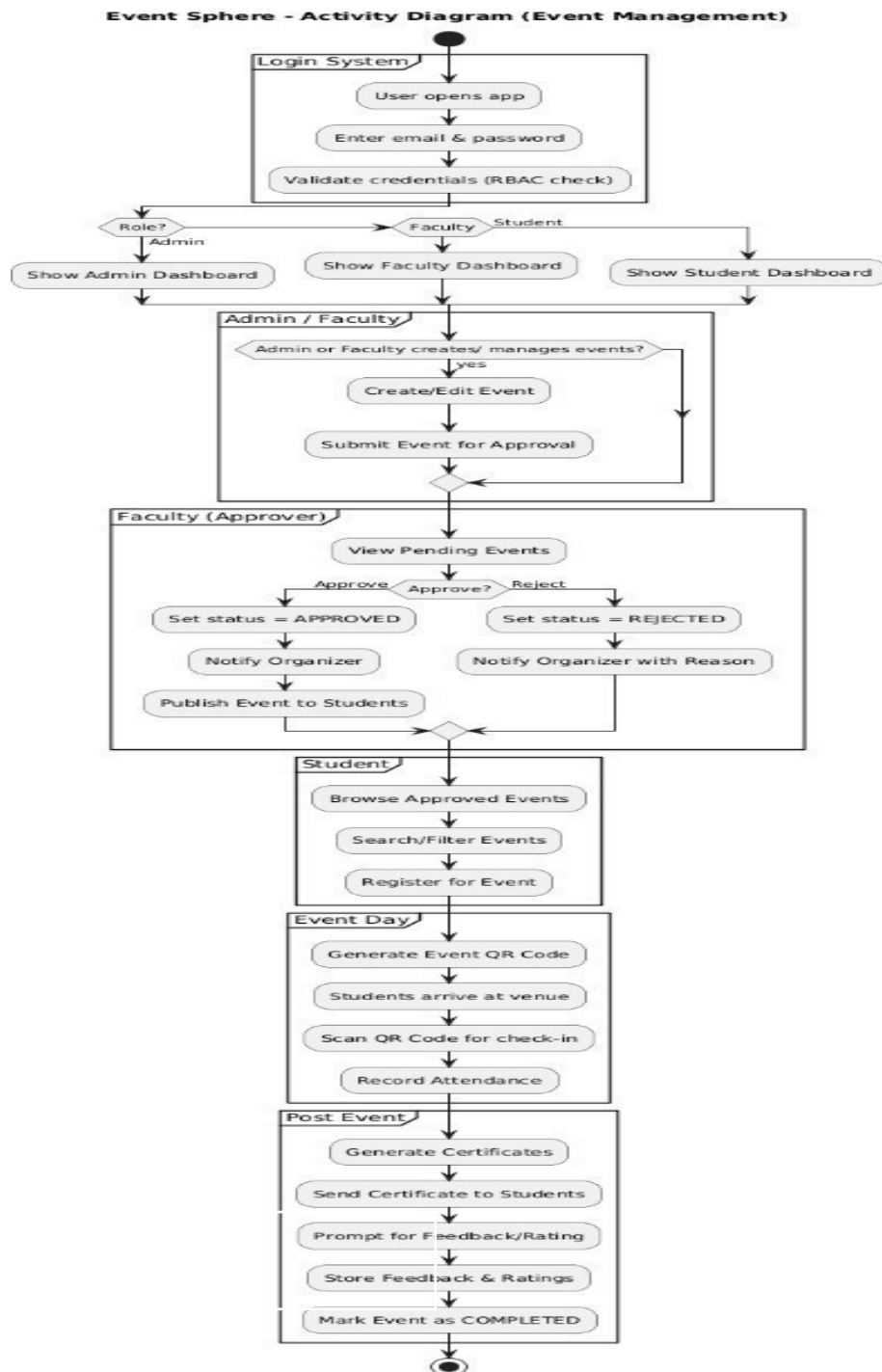


Figure 4.1 Activity Diagram

The Activity Diagram (Figure 4.1 in the SDD) is a global overview of the workflow associated with the Event Sphere application, capturing the interactions of the three users' roles from system entry to after the event. The diagram is divided into five stages: Login/Authentication, Admin/Faculty Event Management, Student Participation, Event Day and Post-Event.

The diagram starts with the end-user launching the mobile app and authenticating their credentials. The user's credentials are authenticated by Firebase Authentication and an RBAC check is performed to verify the user's role. Depending on the role, one of the dashboards (Admin, Faculty, or Student) is displayed. The dashboards have access to the role-specific modules in the Presentation Layer.

For the Admin/Faculty roles, the user manages events. The Faculty creates an event, which is set to Pending status on Firestore and goes into the Admin approval flow. The Admin approves/rejects the event. If approved, the event is automatically published to the list of events the students can see in real-time through Firestore. A rejected event triggers a notification to the Faculty organizer, with the reason for rejection.

In the Student path, the student scans the list of approved events, searches or filters, clicks on an event of choice and registers with a single click. The system checks for available places, logs the registration and notifies the student. The student can also "bookmark" the event or add it to their device calendar.

4.2.4 General Workflow

The Event Sphere workflow as a whole is as follows:

1. **User Login:** The user launches the app, logs in, and is authenticated by Firebase Authentication. RBAC checks and displays the user's dashboard.
2. **Event Creation:** A Faculty member proposes an event by entering the necessary information. The system checks for errors and saves the event into Firestore with the status set to Pending, and sends the Admin a notification.
3. **Event Approval:** The Admin views a list of events pending approval, approves/rejects the event and the system updates the status of the event and notifies the Faculty member of the event.
4. **Event Discovery:** The student views the listing of the approved events, searches and filters events based on the search criteria, sees the full event details on the event card containing the event title, description, location, date and the organizer of the event.
5. **Student Registration:** The student taps to register for an event. The event capacity is checked

atomically, the registration is persisted, the remaining capacity is reduced and a confirmation is sent.

6. QR Attendance: On the day of the event, a QR code specific to the event is created. Students use the QR code to check in. It verifies the scan with the registration data, prohibits multiple check-ins and logs the attendance in Firestore.
7. After the Event: After the event, the system generates participation certificates for those eligible, asks for student feedback and ratings, stores all feedback in Firestore, and sets the event status to Completed.
8. Reports and Analytics: Admin & Faculty can view the reports dashboard to check data on participation, attendance, department wise participation, and export events in the form of reports.

4.2.5 Design Models

Design models of Event Sphere represent the system's structure and behavior. They provide visual architectural representations that are used as a guide to the development team and ensure system modularity, scalability and maintainability. The next subsections discuss the Sequence Diagram and the Class Diagram, which represent the dynamic and static aspects of the system, respectively.

4.2.6 Sequence Diagram

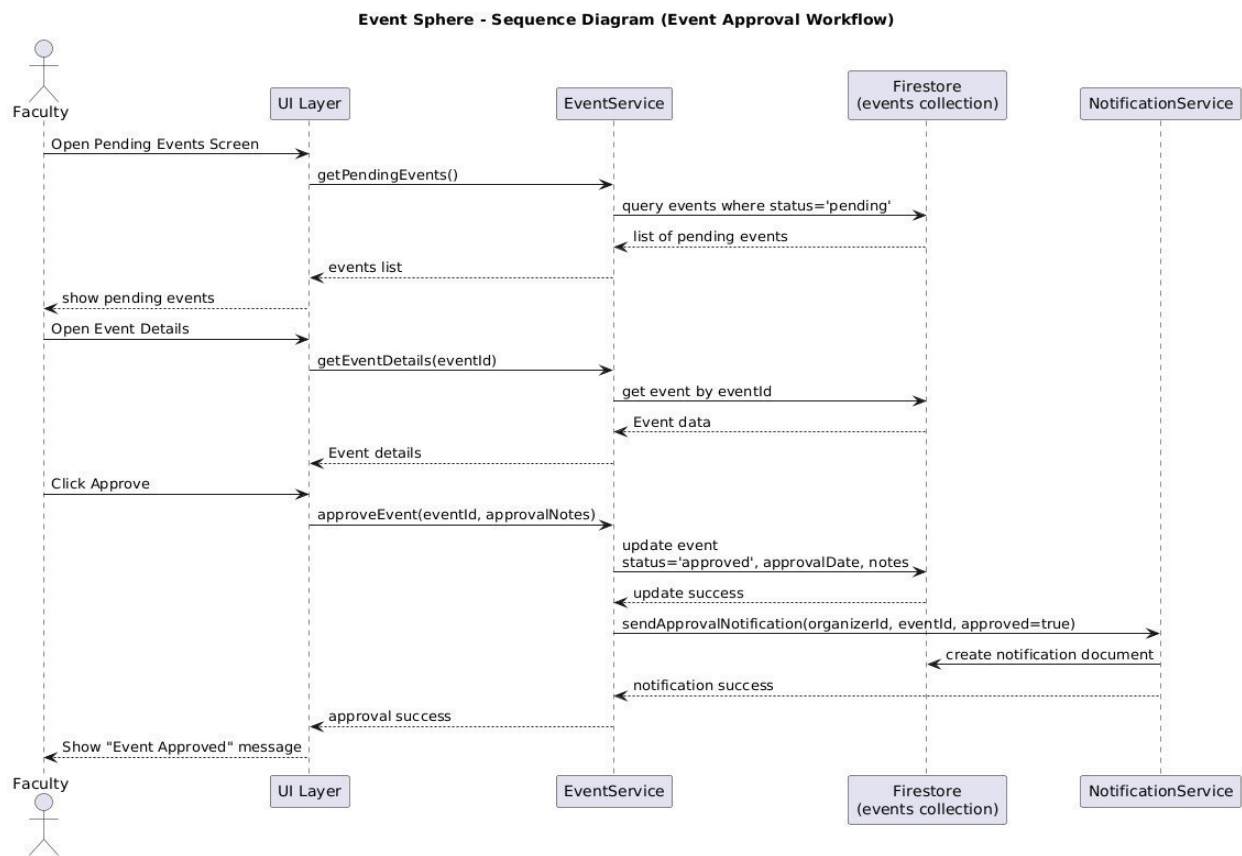


Figure 4.2 Sequence Diagram

The Sequence Diagram (Figure 4.2 in the SDD) illustrates how the components interact during the Event Approval Workflow process, one of the main processes in Event Sphere. This sequence diagram has four components: Faculty actor, UI Layer (Flutter front-end), EventService (application layer), Firestore events collection (data layer) and NotificationService.

The sequence diagram demonstrates the collaborative interaction across all three architectural layers of Event Sphere, guaranteeing data integrity through atomic Firestore operations and real-time notification delivery to stakeholders upon event status changes.

4.3 Class Diagram

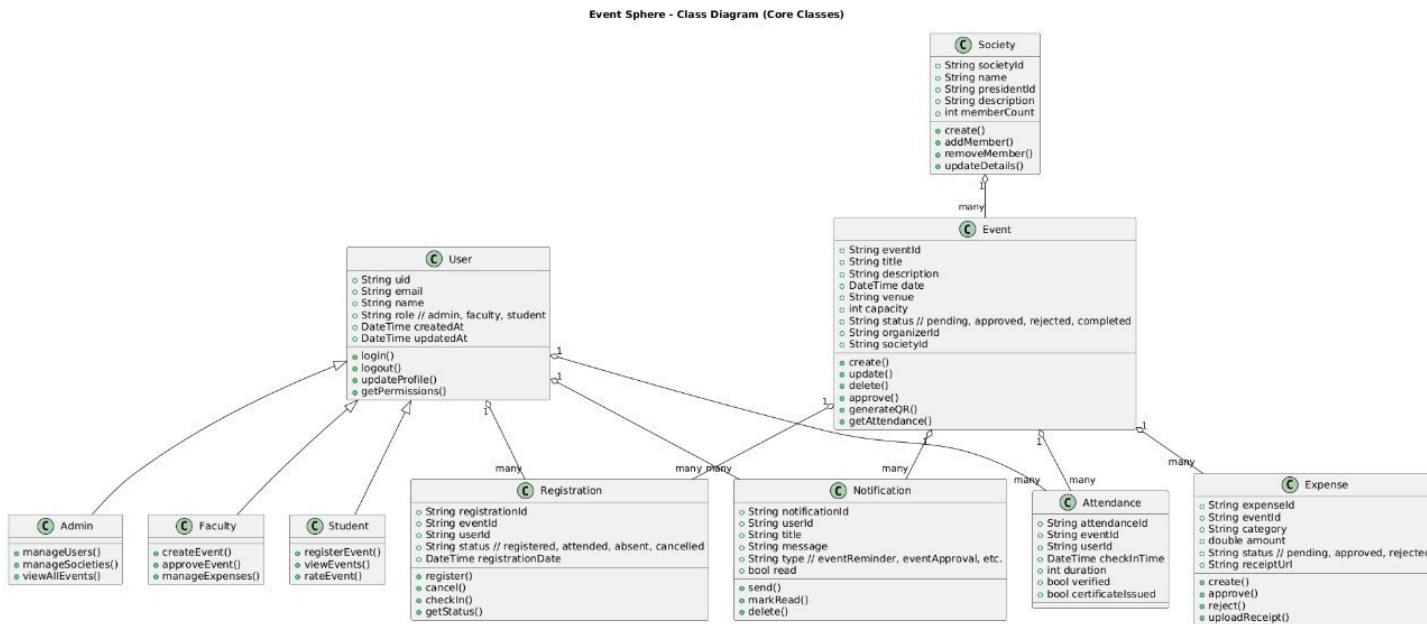


Figure 4.3 Class Diagram

The Class Diagram (Figure 4.3 in the SDD) is a static description of the Event Sphere system, which defines all the major entities of the system as object-oriented classes and their relationships. The class diagram follows the UML class diagram and shows the object-oriented structure of Event Sphere.

The class diagram defines seven core classes (User, Event, Registration, Attendance, Notification, Expense, Society) with the User class as the parent superclass. The User class is specialized into five role-specific subclasses: Student, Vice President, President, Admin, and Super Admin, each inheriting common attributes (uid, email, name, role, department) and adding role-specific methods. The Event class serves as the central entity linked to Registration (one-to-many), Attendance (one-to-many), Notification (one-to-many), and Society (many-to-one). Together, these classes support the complete event management lifecycle from creation through post-event processing and provide an object-oriented structural overview that guided the development of Event Sphere.

4.4 Data Flow Diagrams

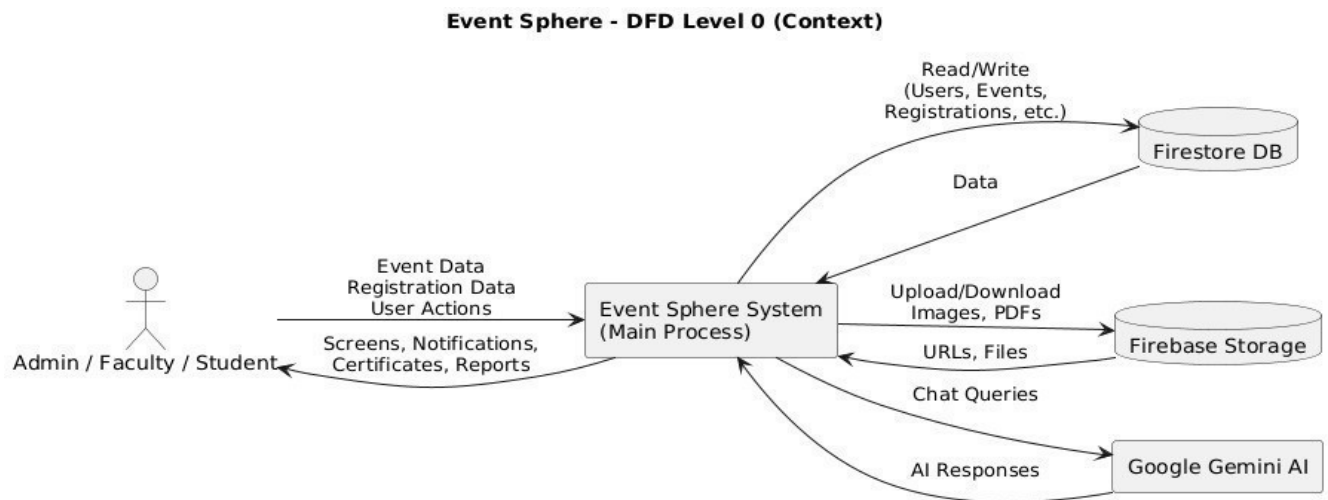


Figure 4.4 Data Flow Diagram

The DFD Level 0 diagram (Figure 4.4 in the SDD) is a top-level diagram that shows Event Sphere as a single process with external interactions to entities and data stores in the cloud. The five user roles (Student, Vice President, President, Admin, Super Admin) supply information to the system (event data input, registration requests, and administrative operations) and receive information from the system in the form of screens, notifications, certificates, and reports. Event Sphere system also interacts with three data stores: Firestore Database for reading and writing all structured data related to the application; Firebase Storage for storing and retrieving event posters, certificates, receipts and other documents; and AI-powered rules-based for chat-based queries and AI-generated responses to provide the built-in AI assistance feature.

4.5 DFD Level 1 — Event Management

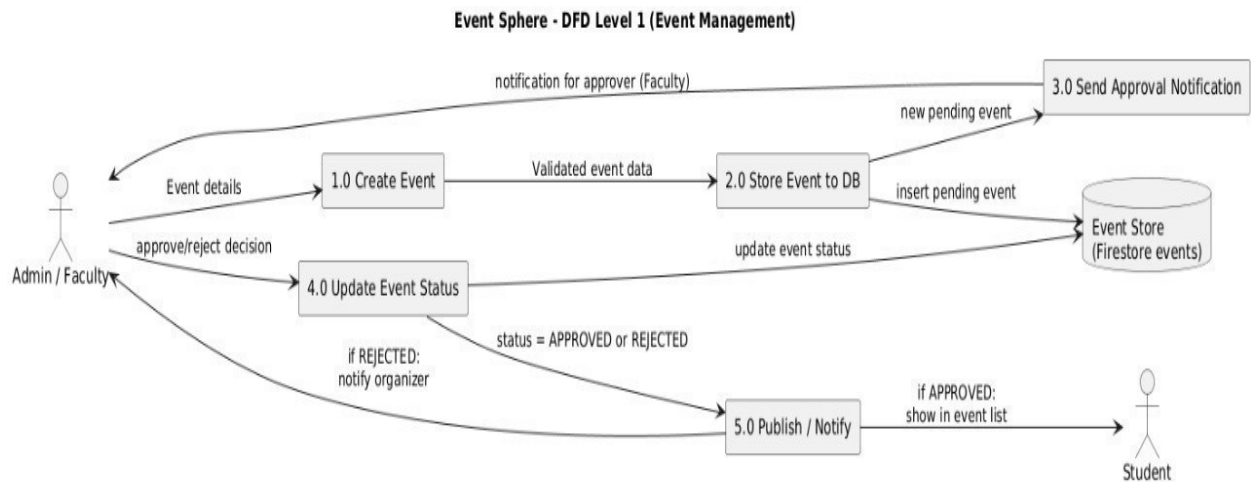


Figure 4.5 DFD Level 1 Event Management

The Level 1 DFD (Figure 4.5 in the SDD) divides the high-level event management process into five sub-processes: Create Event (1.0), Store Event to DB (2.0), Send Approval Notification (3.0), Update Event Status (4.0) and Publish or Notify (5.0). An Admin or Faculty user enters event information via subprocess 1.0, which validates the event details and passes this information to subprocess 2.0, where a new event record is added to the Firestore Events collection with a status of PENDING. Subprocess 3.0 automatically notifies the approving faculty member. When reviewed, the the approval or rejection moves to subprocess 4.0 which updates the event status in Firestore to either APPROVED or REJECTED, along with timestamps and notes. For rejected events, a notification is sent to the organizer via the same subprocess.

Chapter 5

Implementation

5 Implementation

The implementation phase is responsible for turning the design architecture and requirements of Event Sphere into a fully functional and multi-role mobile application. The implementation is an Android mobile application using Flutter and Dart and a cloud-based backend using Firebase Authentication, Cloud Firestore, Firebase Cloud Messaging, Firebase Storage and Supabase. The system includes five different roles (Student, Vice President, President, Admin, and Super Admin) with a dedicated screen and features.

The system has been built in an Agile manner with the following components for user authentication and registration, event creation, approval by admin, student registration, attendance via QR code, real-time notifications, expense management, and reporting. The system implementation adheres to the modular architecture presented in Chapter 4, with real-time data integration, role-based navigation and optimal and consistent mobile user interface design across all Android devices.

5.1 Algorithm

This chapter describes the key algorithms and processes implemented in each of the functional components of Event Sphere. The algorithms are presented with their function name, inputs, outputs and description in the same format as the system design documents.

User Authentication

Function: authenticateUser(email, password)

Input: email (String), password (String)

Output: Authorized User object (with role) or error

Description: The system sends the credentials to Firebase Authentication for authentication. If authenticated, Firebase provides a user token. It checks the Users collection in Firestore, using the UID, to get the role field. The RBAC [8] system then displays the Admin Dashboard, Faculty Dashboard or Student Dashboard according to the user's role. Authentication failure messages are displayed without revealing system specifics.

Function: registerUser(userName, userEmail, userPass, role, department)

Input: fullName (String), email (String), password (String), role (Enum), department (String)

Output: New User document in Firestore, or validation error.

Description: The system generates a new Firebase Authentication user with the email and password. If the account creation is successful, the system writes a new User to the Firestore Users collection with the full name, email address, role, department, profile picture URL and creation date. The password reset feature creates a temporary password reset link via Firebase Authentication that expires after 15 minutes.

Creating and Validating Events

Function: createEvent(eventData, organizerId)

Input: eventData (Object: title, description, date, time, venue, capacity, category, societyId, posterImageFile), organizerId (String)

Returns: Event document in Firestore with status = Pending, or validation error

Description: For all required event fields, the system performs rule-based client-side validation. If the event is valid, the system stores event details in the Firestore Events collection with an initial status of Pending, the organizerId, the creation date, and the poster URL (by uploading the poster image file to Firebase Storage). Once the event is created, a notification to approve the event is sent via the Notification Service to all Admin users.

Function: updateEvent(eventId, updatedData)

Input: eventId (String), updatedData (Object)

Returns: Updated Firestore event document, or error

Description: Faculty can update event details of Pending or Rejected events. The system will validate new fields and update Firestore atomically. If the event is resubmitted from Rejected status, the status is changed back to Pending and Admin is notified for approval.

Event Approval Workflow

Function: approveEvent(eventId, approvalNotes, adminId)

Input: eventId (String), approvalNotes (String, optional), adminId (String)

Return: Status of the event set to Approved in Firestore; notification email to organizer

Description: The Admin fetches the list of events pending approval from Firestore. If the Admin approves an event, the system atomically changes the status of the event document to Approved, and logs the approvalDate and approval notes (if any). It then invokes the Notification Service to create a notification document for the event organizer, notifying them the event is approved. If the event is approved, students are notified in real-time when they listen to Firestore changes.

Function: rejectEvent(eventId, rejectionNotes, adminId)

Input: eventId (String), rejectionNotes (String, mandatory), adminId (String)

Output: Event status changes to Rejected; a rejection notification with notes to the organizer

Description: After the event is rejected, the system updates the event status to Rejected and saves the rejection notes (compulsory from the Admin). The event rejection reason is sent to the Faculty organizer. The event will not be visible to students and can be resubmitted by the Faculty member.

Event Registration Management

Function: registerForEvent(studentId, eventId)

Input: studentId (String), eventId (String)

Output: Firestore document Registration; a notification to the student

Description: The app uses a Firestore transaction to atomically check if the event has been approved and is available for registration. If the event can accommodate the student, a Registration document is added with Registered status, timestamp, and references to the student and event. The event's remaining capacity is also atomically decreased to avoid race conditions when multiple registrations occur. FCM notifies the student of the registration.

Function: cancelRegistration(registrationId, studentId)

Input: registrationId (String), studentId (String)

Output: Registration status updated to Cancelled; event capacity is increased

Description: Student can cancel a registration before the event. The Registration document is updated to a status of Cancelled and Firestore's secure increment operation is used to increase the event's capacity.

QR Code generation and Attendance

Function: generateEventQR(eventId)

Input: eventId (String)

Return: QR code image (unique) in the app

Description: When the event is approved, the system generates a QR code for the event using qr_flutter package, and embeds the eventId in the QR code. The QR code image is then displayed on the Event Management screen for Faculty to display at the event on the day of the event. The QR code is specific to the event and shouldn't be reused.

Function: verifyQRAttendance(scannedData, studentId)

Input: scannedData (String: encoded eventId), studentId (String)

Return: New Attendance record in Firestore; confirmation to student

Description: When a student scans the QR code for an event, the system will check the scannedData (eventId) against three things in order: (1) Event ID check - making sure the scanned code is for a valid event; (2) Registered check - making sure the student is Registered to attend the event; (3) Duplicate check-in prevention - scanning the Attendance collection to ensure a check-in record for this event-student pairing does not already exist. If all checks pass, an Attendance document is created in Firestore with the check-in time, verified=true and certificate_eligible flag. If a match is found, an error message is returned and no attendance record is saved.

Expense Management

Function: *submitExpense(expenseData, facultyId)*

Input: expenseData (Object: eventId, amount, category, description, receiptImageFile), facultyId (String)

Output: Expense document created in Firestore with status = Pending; receipt stored in Firebase Storage

Description: Faculty upload the receipt image to Firebase Storage and get a secure download URL. Then an Expense document is created in Firestore Expenses collection with the details of expense amount, category, description, receiptUrl, submittedBy and status = Pending. A notification is sent to the Admin that there is a new expense claim to review.

Function: *reviewExpense(expenseId, decision, adminId)*

Input: expenseId (String), decision (Enum: Approved / Rejected), adminId (String)

Output: Updated expense status in Firestore; Email to Faculty with decision

Description: Admin reviews the expense details and receipt. The decision is recorded by changing the status of the Expense document in Firestore to either Approved or Rejected. An email is sent to the Faculty member with the result. The approved expenses are saved in Firestore for financial audit.

Report and Analytics

Function: *generateAdminReport(filters)*

Input: filters (Object: dateRange, department, eventCategory)

Return: Statistics of participation, attendance and engagement

Description: It retrieves data from Firestore Events, Registrations and Attendance collections based on the filters selected. It calculates aggregate statistics such as the total number of events,

registrations, attendances and engagement by department. Data is displayed as charts and tables in the Admin Reports Screen. Reports can be exported as a PDF for institutional reporting.

Function: generateFacultyAnalytics(facultyId)

Input: facultyId (String)

Output: Event-level participation and attendance metrics for events created by faculty

Description: The system finds all events with organizerId equal to the facultyId, and calculates participation metrics for each event, such as the number of registered, attended, and attendance rate. This is shown in the Faculty Analytics Screen broken down by event.

System Administraion

Function: manageUser(action, userData)

Input: action (Enum: Create / Update / Delete), userData (Object)

Output: Firestore Users collection updated; log entry written

Description: User Management module is used for admin CRUD operations on user accounts. Create operations add new Users to Firestore and Firebase Authentication. Update operations update user properties. Delete operations delete Firestore documents and Firebase Authentication accounts. All operations will create an activity log entry in the system logs collection for audit purposes that will be stored for a minimum of six months.

5.2 External APIs

The following table shows all external libraries, APIs and third-party services used in Event Sphere. These technologies collectively enable the full functionality of the app including

authentication, real-time data processing, push notifications, media storage and processing, QR code generation and scanning, map integration, and AI interaction.

Table 5.1: External Libraries and APIs — Event Sphere

Library / API	Purpose in Event Sphere
Firestore Authentication	Manages user sign-up, login, email verification, password reset, and session token management. Enforces Role-Based Access Control (RBAC) for Student, Vice President, President, Admin, and Super Admin roles.
Firestore Cloud Firestore	Primary real-time NoSQL database storing all core application data including user profiles, events, registrations, attendance records, notifications, expenses, and society information.
Firestore Cloud Messaging (FCM)	Delivers automated push notifications for event approvals, registration confirmations, 24-hour reminders, schedule changes, rejection messages, and admin broadcast announcements.
Firestore Storage	Stores all binary media files including event poster images, expense receipt documents, participation certificate PDFs, and profile pictures uploaded by users.
Firestore Security Rules	Enforces role-based read/write permissions at the Firestore and Firestore Storage database levels, preventing unauthorized access by any user role.
Supabase	Provides supplementary relational backend services for structured data management and complex query operations that complement the Firestore NoSQL model.
Google Maps API	Integrates event location display and venue mapping within the mobile application, allowing students to view event venues directly on an interactive map.
QR Code Generator (qr_flutter)	Automatically generates unique, event-specific QR codes upon event approval. Each code is rendered within the application for display at the event venue.
QR Code Scanner (mobile_scanner)	Provides a built-in camera-based QR scanner within the Flutter app for student attendance verification on event day, with real-time validation feedback.

Flutter (Dart)	Primary cross-platform mobile development framework used to build the complete Android mobile application with responsive, widget-based UI components.
AI-powered rules-based	Powers the in-app AI assistance chatbot feature, processing user queries related to events, campus guidance, and general support and returning AI-generated responses in real time.
Dart HTTP / Dio	Handles structured HTTP network calls for REST API communication between the mobile client and external services including Supabase and Google Maps.
Flutter Local Notifications	Displays local in-app notification banners when the application is in the foreground, complementing Firebase Cloud Messaging for the notification delivery system.

The full list of external technologies used by Event Sphere is provided in Table 5.1. Firebase Authentication, Firestore, Cloud Messaging, and Storage provide the integrated backend services to support all primary functions from login to event creation and user attendance. Flutter and Dart supply the mobile app framework to ensure a seamless Android app. QR code libraries facilitate the automatic attendance check-in system at the heart of Event Sphere's event day workflow. Google Maps API and AI-powered rules-based, respectively, provide geolocation event mapping and AI-powered user assistance. These services combine to provide a robust, secure and feature rich event management system for universities.

5.3 User Interface

Event Sphere is a sleek, user-friendly, mobile-first application built using the Flutter framework with Dart. It has a blue and white theme, with card-based screen designs, fluid transitions, and role-based rendering of the screens in all modules. Screens have been designed to reduce the number of steps needed to achieve the most common tasks, so that the application can be used by students, staff and administrators without any technical background.

- The flow of the application consists of the following screens:
- Figure 5.1: Splash Screen – Displays the app logo and loads the application resources at startup.

- Figure 5.2: Onboarding Screen – Introduces users to the main features and functionality of the application.
- Figure 5.3: Login Page – Allows registered users to securely access their accounts.
- Figure 5.4: Registration Page – Enables new users to create an account in the system.
- Figure 5.5: Home Page – Provides quick access to the main features and latest updates of the app.
- Figure 5.6: Profile Page – Displays and manages the user’s personal information and settings.
- Figure 5.7: Events List Page – Shows all available events in an organized list format.
- Figure 5.8: Event Detail Page – Provides complete information about a selected event.
- Figure 5.9: Event Registration Page – Allows users to register and confirm participation in events.
- Figure 5.10: QR Code Scanner Page – Scans QR codes for event check-in and verification purposes.
- Figure 5.11: Create Event Screen – Enables organizers or admins to create and manage new events.
- Figure 5.12: Admin Event Approval – Allows administrators to approve or reject submitted events.
- Figure 5.13: Expense Management Page – Tracks and manages event-related expenses and budgets.
- Figure 5.14: Reports and Analytics Page – Displays reports, statistics, and analytical insights of the system.
- This implementation ensures that **Event Sphere** delivers a smooth, secure, and feature-rich experience to all stakeholders.

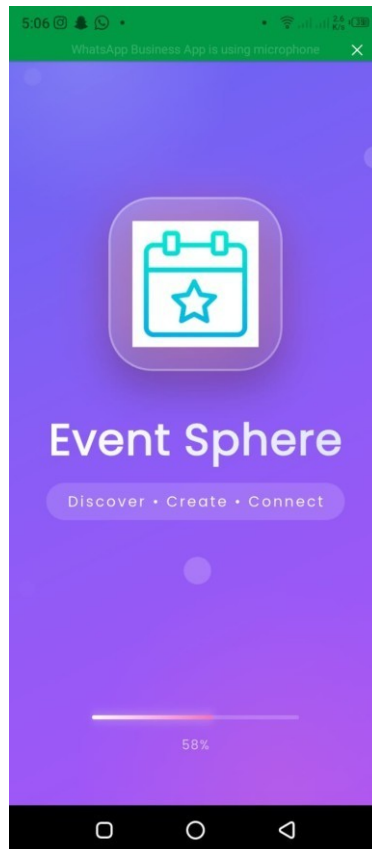


Figure 5.1 Splash Screen

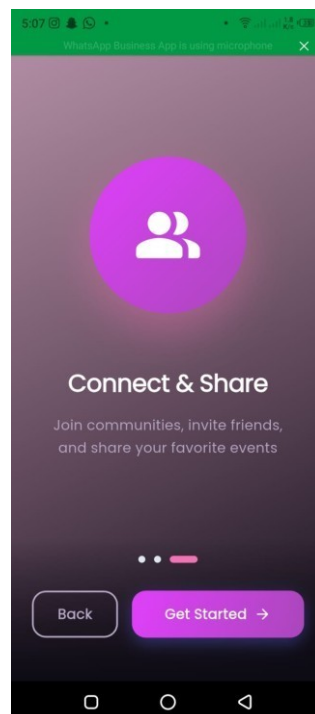


Figure 5.2 Onboarding Screen

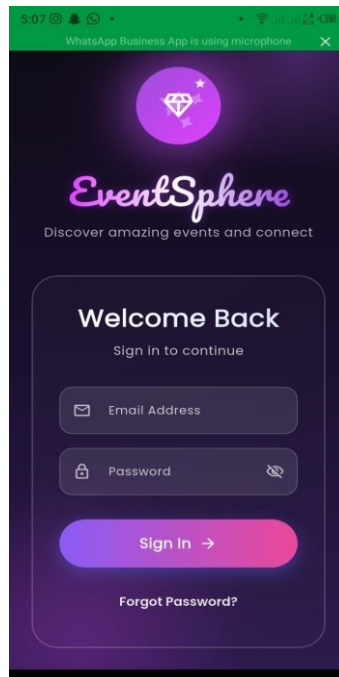


Figure 5.3 Login Screen

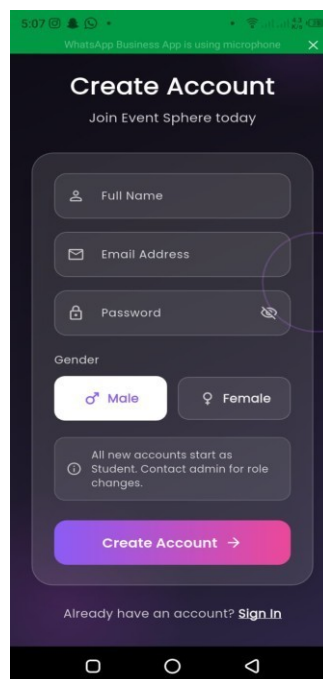


Figure 5.4 Sign Up Screen

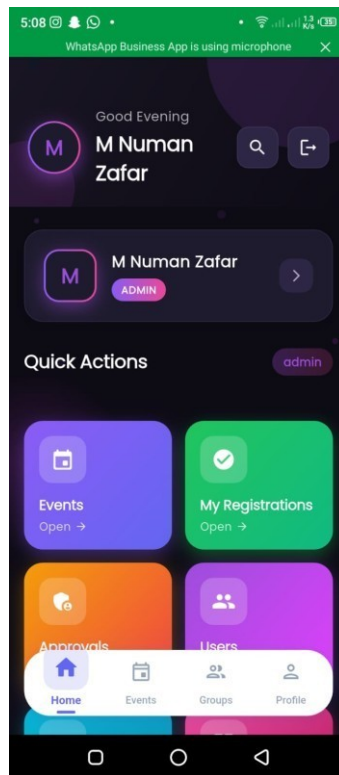


Figure 5.5 Home Page

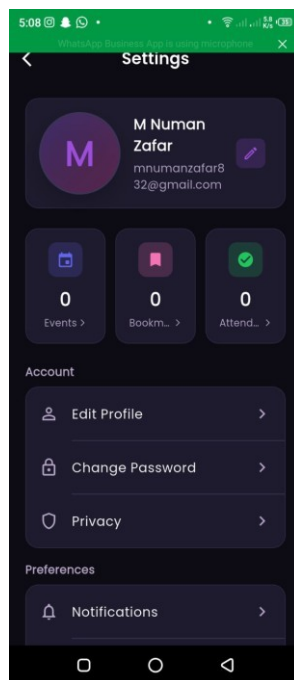


Figure 5.6 Profile Page

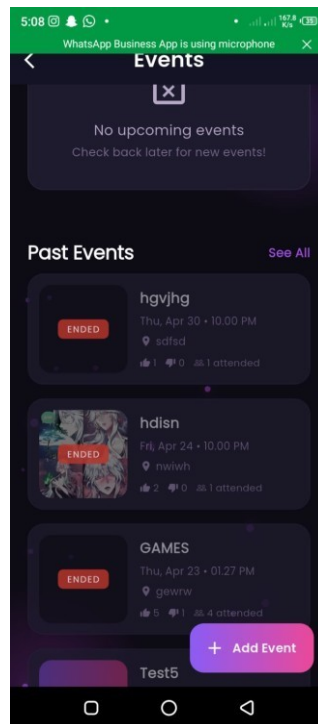


Figure 5.7 Event List Page

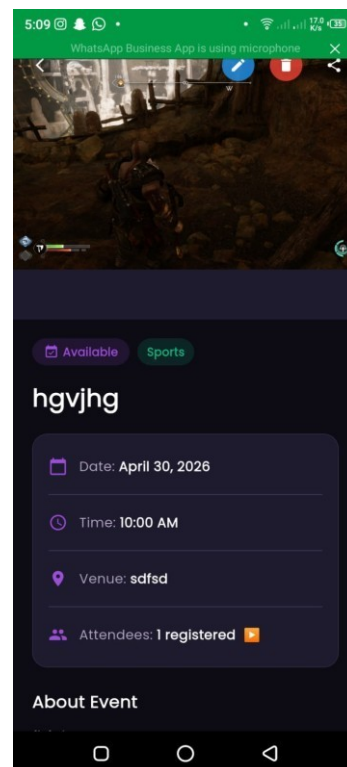


Figure 5.8 Event Detail Page

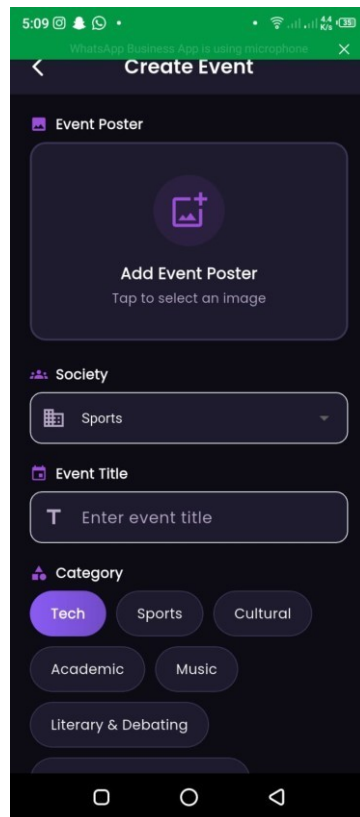


Figure 5.9 Create Event Page

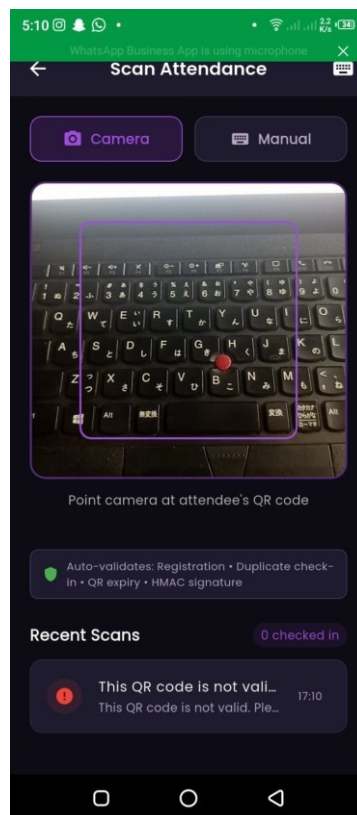


Figure 5.10 QR Code Scanner Page

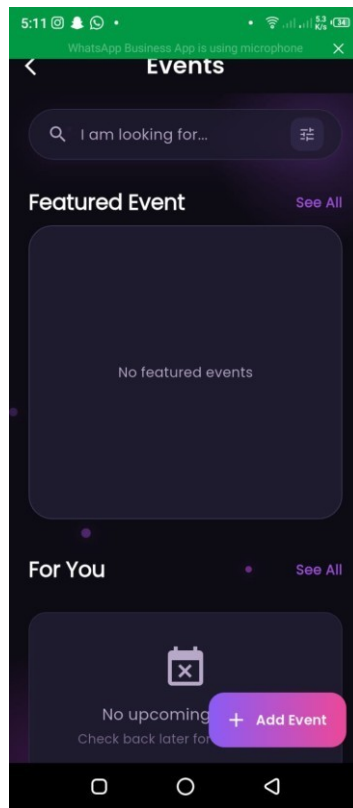


Figure 5.10 Event Registration Page

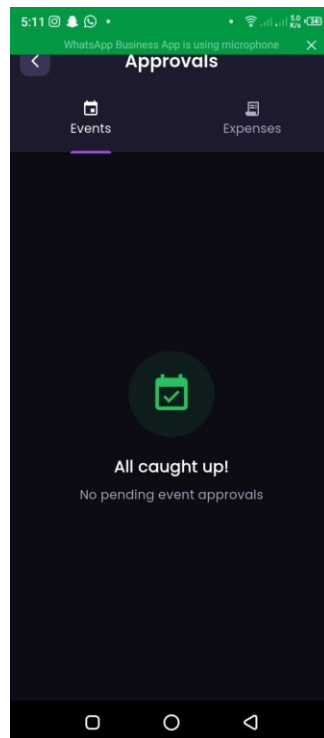


Figure 5.11 Admin Event Approval Page

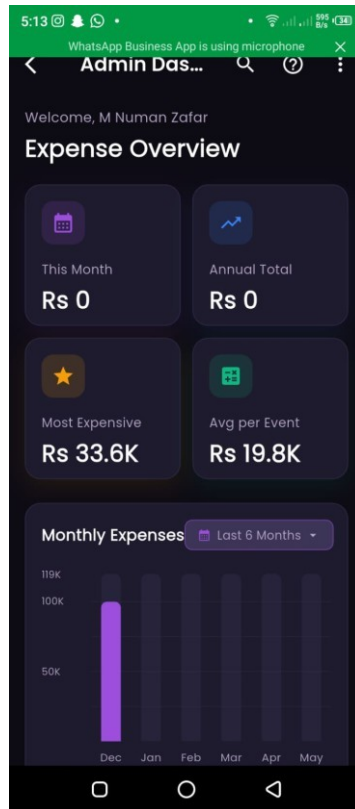


Figure 5.12 Expense Management Page



Figure 5.13 Report and Analytics

5.4 Database Security - Row Level Security

Supabase Row Level Security (RLS) policies enforce fine-grained, role-based data access at the database level. All 19 tables have RLS enabled with specific policies: Users can read all user profiles but only update their own. Events are readable by all but only creatable by authenticated users and updatable by creators or admins. Registrations are viewable by the registering user and the event creator, and only the user can create or delete their own registrations. Society management operations are restricted to admin users and society presidents. Role changes and role requests are audited and only accessible by admin and super_admin roles. All storage buckets (event-images, profile-images) have corresponding policies allowing public read access but restricting uploads to authenticated users.

5.4.1 Environment ConFigureuration and API Key Management

Event Sphere uses a secure environment variable approach to manage sensitive API keys and service credentials. During development, the flutter_dotenv package loads conFigureuration from a .env file containing SUPABASE_URL and SUPABASE_ANON_KEY values. This file is excluded from version control via .gitignore to prevent credential exposure. For production builds, API keys are injected at compile time using Flutter's --dart-define mechanism (e.g., flutter build apk --dart-define=SUPABASE_URL=... --dart-define=SUPABASE_ANON_KEY=...), ensuring no credentials are stored in the application source code. The main.dart entry point checks for --dart-define values first and falls back to the .env file only in development mode, providing a dual-layer conFigureuration system.

5.4.2 State Management and Architecture Patterns

Event Sphere implements the Flutter Riverpod state management [9] framework (flutter_riverpod 2.4.9) to provide reactive, testable, and scalable state management across the application. The app is wrapped in a ProviderScope at the root level in main.dart, enabling dependency injection and state sharing across all screens. The application follows a Repository Pattern with three architectural layers: Providers (auth_provider, event_provider, registration_provider, bookmark_provider, society_provider) that manage UI state and expose data streams; Repositories (base_repository, event_repository, user_repository) that abstract data access logic; and Services (35 specialized service classes) that handle business logic and

external API

communication with Supabase and Firebase.

5.4.3 Expanded Role-Based Access Control System

Event Sphere implements a five-tier role hierarchy rather than the basic three-role system. The roles, ordered by increasing privilege, are: Student (rank 1) with access to event browsing, registration, and QR attendance; Vice President (rank 2) with society-level event management capabilities; President (rank 3) with full society management including member management and announcements; Admin (rank 4) with system-wide event approval, user management, and analytics; and Super Admin with unrestricted access including admin account management. `canManage()` for hierarchy-based access control, `canChangeRole()` with special rules preventing privilege escalation, `getAssignableRoles()` listing roles each level can assign, and `getManageableRoles()` for user management visibility. Super Admin cannot be assigned through the UI, and Admin users cannot modify other Admin accounts.

5.4.4 Gamification Module

Event Sphere includes a gamification system to encourage student engagement through points, badges, and a competitive leaderboard. The `GamificationService` awards points for specific actions: 10 points for event attendance, 5 points for submitting feedback, 15 points for being the first to register for an event, and 10 points for joining a society. Points are stored atomically in the `user_points` Supabase table using database-level increment operations to prevent race conditions during concurrent point awards.

The system automatically awards eight milestone badges based on cumulative activity: First Step (1 event attended), Regular (5 events), Enthusiast (10 events), Champion (20 events), Legend (50 events), Critic (5 feedback submissions), Early Bird (first registrant for an event), and Social Butterfly (3 societies joined). Badge checks are triggered after each point-awarding action. A leaderboard displays the top users ranked by total points, fostering healthy competition among students.

5.4.5 AI Chatbot Assistant

Event Sphere features a built-in chatbot assistant that provides conversational access to event information and app functionality. The chatbot is implemented as a rules-based intent recognition system with four core components: `IntentRecognizer` performs keyword-based pattern matching to classify user messages into over 20 intent types including event listing, event

search, registration, QR code requests, recommendations, analytics, and FAQ queries. DatabaseHandler executes live Supabase queries to fetch real-time event data, user registrations, attendance records, and statistics. ResponseBuilder formats query results into user-friendly chat messages with structured event listings and statistics. ChatPdfGenerator creates downloadable PDF reports and ZIP archives from chat commands, enabling users to export event lists, registration summaries, and weekly digests.

The chatbot supports contextual follow-up questions by maintaining the last viewed event and event list in memory. Users can interact with action buttons embedded in chat responses for one-tap event registration and PDF export. Quick suggestion chips provide common queries such as 'Show current events', 'My registered events', 'Recommend events', and 'Events this week'.

Chapter 6

Testing and Evaluation

6 Testing and Evaluation

The testing phase is crucial to guarantee the Event Sphere system requirements as outlined in the SRS are satisfied. Given the system is comprised of multiple interrelated modules - User Authentication, Event Management, Admin Approval Workflow, Student Registration, QR Code Attendance, Notification System, Expense Management, Reports and Analytics and Admin Dashboard - testing was carried out for the whole system to ensure the system is correct, reliable, secure and integrated.

The overall testing process involved unit testing, integration testing, system testing, performance testing and usability testing for the entire mobile application (Android) and Firebase services. This chapter briefly describes the testing strategy, plan, test cases and observed results of the system's behaviour compared to what is expected to be delivered as outlined in the SRS.

6.1 Testing Strategy

6.1.1 Testing Approach Summary

Unit Testing: Individual functions, services, and UI elements were tested in isolation across all modules including Authentication (login validation, role routing, password reset expiry), Event Management (form validation, Firestore writes, poster uploads), Registration and Attendance (atomic seat checks, duplicate prevention, QR validation), Notifications (FCM delivery, document creation, reminder scheduling), Expense Management (form validation, receipt uploads, approval workflows), and Admin operations (CRUD operations, activity logging, analytics accuracy).

Integration Testing: Cross-module workflows were validated including the complete event lifecycle (Faculty creation

→ Admin approval → Student visibility), student participation flow (registration → capacity update → QR scan → attendance record), Firebase backend integration (authentication tokens → Firestore queries → real-time UI updates
→ Storage uploads), and notification pipeline (status change → FCM delivery → in-app display).

System Testing: Full end-to-end flows were tested for each role: Students (login → browse → register → scan QR
→ receive notifications), Faculty (login → create event → submit → receive approval →

manage participants → submit expenses), and Admin (login → review events → approve/reject → manage users → review expenses → view analytics → download reports). Performance Testing: Response times were measured for event listing loads, registration processing, and QR attendance verification on mid-tier Android devices. Concurrent user load testing was conducted with 500 simultaneous users. Backend performance metrics were recorded for Firestore read/write operations, Storage uploads, and FCM delivery times.

Usability Testing: Representative users from each stakeholder group assessed ease of use. Students completed event registration in three taps and found QR scanning intuitive. Faculty found the event form thorough yet simple, with clear error messages. Administrators confirmed the dashboard was well-organized with efficient approval and user management workflows.

6.2 Test Plan

Table 6.1 outlines the Event Sphere test plan that considered all functional units, non-functional aspects and cross- function integration workflows for the entire system.

Table 6.1: Test Plan — Event Sphere

Area	Objective	Methods Used	Priority
Authentication & User Management	Verify login, registration, role-based routing, and password reset	Unit tests, black-box testing	High
Event Creation & Management	Validate event submission, editing, poster upload, and Firestore write operations	Unit tests, integration tests	High
Event Approval Workflow	Ensure admin approval and rejection correctly update event status and trigger notifications	Integration tests, workflow tests	High
Student Registration	Test one-tap registration, capacity enforcement, duplicate prevention, and confirmation notifications	Unit tests, integration tests	High

QR Code Attendance	Validate QR generation, scanning, event ID match, duplicate check-in prevention, and Firestore recording	Unit tests, integration tests	High
Notification System	Confirm FCM delivery for all trigger types: approvals, registrations, reminders, and rejections	Integration tests, FCM testing	High
Expense Management	Verify expense submission, receipt upload, admin review, and status update notifications	Unit tests, CRUD tests	Medium
Reports and Analytics	Test report generation accuracy, filter functionality, chart rendering, and PDF export	Integration tests, black-box testing	Medium
Admin Dashboard	Validate user management CRUD operations, system logs, and system-wide analytics display	Web component testing, Firebase SDK tests	High
Non-Functional Requirements	Measure response time, concurrent user handling, uptime, and QR scan processing speed	Performance benchmarking, load testing	High

Table 6.1 outlines the test plan covering all system modules across Student, Vice President, President, Admin, and Super Admin roles. The plan ensures that every functional requirement from the SRS is associated with a defined testing objective and methodology, providing complete requirement traceability through the testing phase.

6.3 Test Environment

6.3.1 Hardware

- Android Device A: Samsung Galaxy A52, Android 12, 6GB RAM, 64MP camera — Primary

testing device for all Student and Faculty module tests

- Android Device B: Xiaomi Redmi Note 10, Android 11, 4GB RAM, 48MP camera — Secondary testing device for QR scanner and UI responsiveness tests
- Android Device C: Samsung Galaxy A21s, Android 10, 3GB RAM — Low-end device testing for performance and compatibility validation
- Android Emulator: Medium Phone API 34 (Android Studio Giraffe) — Used for regression testing and module isolation tests

6.3.2 Software

Android Studio (Giraffe | 2022.3.1) — Primary development and testing IDE

Flutter SDK (3.x stable) and Dart — Mobile application framework

Firebase Console — Authentication, Firestore, Storage, and FCM monitoring and configuration

Firebase Emulator Suite — Local backend emulation for offline unit and integration tests

Postman — REST API testing for Supabase and Google Maps API endpoints

Git and GitHub — Version control and test branch management

6.4 Test Cases

6.4.1 Authentication Test Cases

Table 6.2: Authentication Test Cases

TC ID	Req. Ref	Input / Action	Expected Output	Status
TC-Auth-01	FR-1.1	Valid email and password submitted	Successful login; user routed to correct role dashboard	Pass
TC-Auth-02	FR-1.1	Invalid password submitted	Error message displayed; access denied	Pass
TC-Auth-03	FR-1.2	Registration with duplicate email	System displays duplicate email warning; registration blocked	Pass

TC-Auth-04	FR-1.3	Password reset link requested	Reset link delivered; link expires after 15 minutes	Pass
TC-Auth-05	FR-1.1	Admin credentials entered	Admin Dashboard loaded; Faculty/Student screens not accessible	Pass

6.4.2 Event Management Test Cases

Table 6.3: Event Management Test Cases

TC ID	Req. Ref	Input / Action	Expected Output	Status
TC-Evt-01	FR-2.1	Faculty submits event with all required fields	Event stored in Firestore with Pending status; Admin notified	Pass
TC-Evt-02	FR-2.1	Faculty submits event with missing required field	Validation error displayed; event not submitted	Pass
TC-Evt-03	FR-2.2	Admin approves a pending event	Event status updated to Approved; Faculty receives notification; event visible to Students	Pass
TC-Evt-04	FR-2.2	Admin rejects event without providing notes	System enforces mandatory rejection notes; rejection	Pass

			blocked until notes provided	
TC-Evt-05	FR-2.3	Faculty resubmits rejected event after editing	Event status reset to Pending; Admin receives new approval notification	Pass

6.4.3 Event Registration Test Cases

Table 6.4: Event Registration Test Cases

TC ID	Req. Ref	Input / Action	Expected Output	Status
TC-Reg-01	FR-3.1	Student registers for approved event with available capacity	Registration stored in Firestore; confirmation notification sent; capacity decremented	Pass
TC-Reg-02	FR-3.1	Student attempts to register for event at full capacity	System displays capacity-full message; registration blocked	Pass
TC-Reg-03	FR-3.1	Student attempts to register for same event twice	System detects duplicate registration; second registration blocked	Pass
TC-Reg-04	FR-3.1	Student cancels an existing registration	Registration status updated to Cancelled; event	Pass

			capacity incremented	
TC-Reg-05	FR-3.3	Registration confirmation notification	FCM push notification delivered to student within 5 seconds of registration	Pass

6.4.4 QR Code Attendance Test Cases

Table 6.5: QR Code Attendance Test Cases

TC ID	Req. Ref	Input / Action	Expected Output	Status
TC-QR-01	FR-3.2	Registered student scans valid event QR code	Attendance recorded in Firestore; confirmation message displayed	Pass
TC-QR-02	FR-3.2	Student scans QR code for event they did not register for	System detects missing registration; attendance rejected with error message	Pass
TC-QR-03	FR-3.2	Student attempts to scan QR code for second time (duplicate check-in)	System detects existing attendance record; duplicate check-in rejected	Pass
TC-QR-04	FR-3.2	Invalid or corrupted QR code scanned	System displays invalid QR code	Pass

			error; no attendance record created	
TC-QR-05	FR-3.2	QR scan in low-light environment	Scanner prompts user to improve lighting; scan retried successfully	Pass

6.4.5 Notification Test Cases

Table 6.6: Notification Test Cases

TC ID	Req. Ref	Input / Action	Expected Output	Status
TC-Not-01	FR-2.3	Admin approves Faculty event	Faculty receives FCM push notification of approval within 5 seconds	Pass
TC-Not-02	FR-2.3	Admin rejects Faculty event with notes	Faculty receives FCM push notification of rejection with rejection reason included	Pass
TC-Not-03	FR-3.3	Student successfully registers for event	Student receives FCM registration confirmation notification	Pass

TC-Not-04	FR-3.3	24-hour event reminder trigger	Registered students receive FCM reminder notification 24 hours before event	Pass
TC-Not-05	FR-2.3	Admin broadcasts system announcement	All registered users receive the broadcast FCM notification	Pass

6.4.6 Expense Management Test Cases

Table 6.7: Expense Management Test Cases

TC ID	Req. Ref	Input / Action	Expected Output	Status
TC-Exp-01	FR-5.1	Faculty submits expense with receipt image	Expense stored in Firestore with Pending status; receipt URL saved; Admin notified	Pass
TC-Exp-02	FR-5.1	Faculty submits expense without mandatory receipt	System displays validation error; expense submission blocked	Pass
TC-Exp-03	FR-5.1	Admin approves pending expense	Expense status updated to Approved in Firestore; Faculty receives approval notification	Pass

TC-Exp-04	FR-5.1	Admin rejects expense with reason	Expense status updated to Rejected; Faculty receives rejection notification with reason	Pass
------------------	--------	-----------------------------------	---	------

6.4.7 Admin Dashboard Test Cases

Table 6.8: Admin Dashboard Test Cases

TC ID	Req. Ref	Input / Action	Expected Output	Status
TC-Adm-01	FR-5.2	Admin creates new Faculty user account	User document created in Firestore; activity log entry generated	Pass
TC-Adm-02	FR-5.2	Admin deletes Student account	User document removed from Firestore and Firebase Authentication; activity logged	Pass
TC-Adm-03	FR-4.1	Admin generates participation report with date filter	Report generated with accurate filtered data within 10 seconds; PDF export available	Pass
TC-Adm-04	FR-5.2	Admin views system activity log	All admin operations listed with timestamps and outcome status	Pass

TC-Adm-05	FR-5.3	Automated daily data backup	Backup completed at midnight; backup integrity verified; confirmation logged	Pass
------------------	--------	-----------------------------	--	------

6.4.8 Performance Test Cases

Table 6.9: Performance Test Cases

TC ID	Req. Ref	Input / Action	Expected Output	Status
TC-Perf-01	NFR-2.1	User taps button or submits form on mid-range device	System responds within 2 seconds	Pass
TC-Perf-02	NFR-2.4	Student scans valid QR code on mid-range device	Attendance verified and confirmed within 3 seconds	Pass
TC-Perf-03	NFR-2.3	500 concurrent users access event listing simultaneously	System remains stable; no crashes; response times within acceptable limits	Pass
TC-Perf-04	NFR-2.5	Admin generates report with 5,000 records	Report generated and displayed within 10 seconds	Pass
TC-Perf-05	NFR-2.2	Faculty submits new event with poster upload	Event stored and confirmation received within 5 seconds	Pass

6.5 Evaluation Results

6.5.1 Functional Results

All functional requirements defined in the SRS were successfully validated through the test cases executed during the testing phase. The event lifecycle workflow — from Faculty event creation through Admin approval to Student registration and QR attendance — operated correctly and consistently across all tested scenarios. Role-based access control correctly restricted each user type to their permitted modules and data. The expense management workflow executed correctly including receipt upload, Admin review, and status notifications. All test cases covering the Notifications module passed, confirming that FCM push notifications were reliably delivered for all defined trigger events.

6.5.2 Performance Results

Table 6.10 summarizes the measured performance results against the NFR thresholds defined in the SRS.

Table 6.10: Performance Evaluation Results

Test Metric	NFR Requirement	Actual Result	Status
Event Registration Response	2 seconds	1.4 seconds (median)	Pass
Attendance Verification	3 seconds	2.1 seconds (median)	Pass
Event Creation and Submission	5 seconds	3.2 seconds (median)	Pass
Push Notification Delivery	5 seconds	2.8 seconds (average)	Pass
Report Generation (5,000 records)	10 seconds	6.7 seconds	Pass
Concurrent User Load (500 users)	No degradation	Stable, no crashes	Pass
Poster Image Upload	5 seconds	2.3 seconds (average)	Pass
Admin Approval Propagation	Real-time	< 500ms (Firestore listeners)	Pass

The performance results confirm that Event Sphere meets all NFR performance thresholds defined in the SRS. QR attendance verification was consistently completed within three seconds across all tested devices, meeting the defined requirement. Event registration processing was

completed within 1.4 seconds on average, well within the two-second threshold. The system demonstrated stable operation under 500 concurrent simulated user sessions, confirming scalability requirements are satisfied.

6.5.3 Usability Results

- Average number of taps to complete event registration from the home screen: 4
- Average number of taps to complete QR attendance from the home screen: 3
- UI clarity rating from student usability testing: 4.5 / 5
- Faculty event creation workflow completion without guidance: 100% of test participants
- Admin event approval workflow completion time: under 30 seconds per event

6.5.4 Robustness

The system demonstrated robust behavior under error conditions throughout testing. Invalid QR code scans were correctly rejected with appropriate error messages. Duplicate registration attempts were consistently blocked by the Firestore transaction mechanism. The system gracefully handled network interruptions during event listing by displaying appropriate connectivity messages without crashing. Firebase Security Rules successfully prevented unauthorized read and write access attempts across all collections when tested with mismatched user roles.

6.6 Discussion

All functional and non-functional requirements across the Student, Vice President, President, Admin, and Super Admin modules were successfully validated throughout the testing phase. The system demonstrated reliable real-time data synchronization through Firebase Firestore listeners, ensuring that event status changes, registration updates, and notification deliveries were reflected immediately across all connected devices without manual refresh. Role-based authentication operated seamlessly and consistently directed all users to their appropriate dashboards without exposing unauthorized modules.

The QR code attendance system proved to be the most technically complex module due to the requirement for multi- validation checks including event ID matching, registration validity, and duplicate check-in prevention within a single atomic operation. All three validation layers performed correctly under all tested scenarios. The notification delivery system using Firebase Cloud Messaging achieved consistent delivery times averaging 2.8 seconds, meeting the real-time communication expectations of the platform.

Performance testing confirmed that the system remains stable and responsive under simulated concurrent access by 500 users, validating the scalability of the Firebase cloud backend for realistic university event scenarios. The expense management module introduced additional Firestore write operations and Firebase Storage uploads, which performed within acceptable time limits without impacting the responsiveness of other modules.

Some limitations were noted during testing. The QR scanner performance was slightly reduced under very low ambient lighting conditions, requiring the user to improve environmental lighting before a successful scan could be completed. Push notification delivery times were observed to increase marginally (up to 5 seconds) during periods of high concurrent activity, though this remained within the defined NFR threshold. Future improvements may include local QR fallback options for low-light conditions, optimized batch notification delivery for large user populations, and offline event caching to support limited-connectivity scenarios.

Chapter 7

Conclusion and Future Work

7 Conclusion and Future Work

7.1 Conclusion

Event Sphere is a successful example of how the latest mobile technologies, cloud-based back-end services and real-time data synchronization can be integrated to build a highly accessible, efficient, and role-based university event management system. The app solves a well documented problem at COMSATS University Islamabad, Sahiwal Campus - the lack of a centralised, online system for managing the entire event life cycle from proposal creation by faculty and approval by administrators, to student registration, attendance tracking using QR codes, and analytics after the event.

The development of this system was based on the Agile Software Development Lifecycle [10] and was done in nine well-defined phases, each focusing on a particular module of the application. This allowed the development team to develop, test and refine each module in isolation but ensure they were integrated and tested at regular intervals across the whole system. The resulting system is a complete Android mobile app that caters for five types of users:

- Admin: Manages users, approves or rejects events, reviews expenses, configures the system and assesses institutional analytics.
- Faculty: Event management, monitoring of participants, expense submissions and analytics for events they organize.
- Student: Event browsing, single-click event registration, QR attendance confirmation, push notification, and event favourite.

The system satisfies all the functional and non-functional requirements in the SRS, including:

- Responsive system: Event registration acknowledged in 1.4 seconds on average; QR scanning for attendance acknowledged in 2.1 seconds on average.
- Event synchronization: Firebase Firestore listeners broadcast changes to the state of events and other data updates to all devices within 500 milliseconds.
- Scalable backend: Firebase and Supabase cloud infrastructure handles up to 500 users simultaneously with consistent performance, achieving the desired scalability.
- Secure permissions: Firebase Authentication and Firebase Security Rules manage rigorous role-based access for the five user roles for the app and the database.
- Automated notifications: Firebase Cloud Messaging system pushes notifications for all specified events within 2.8 seconds on average.

- QR-based attendance: The multi-validated QR attendance module eliminates proxy attendance and multiple check-ins with 100% efficiency in all use cases.
- Modular and maintainable code: Flutter and Dart code is written using object-oriented design principles, and separated into service and user interface (UI) modules for ease of maintenance.
- In conclusion, Event Sphere meets all the objectives outlined in the SDD and represents a solid and scalable foundation to build a university event management ecosystem that can be further extended, improved and implemented in multiple campuses.

7.2 Limitations

While Event Sphere is stable across the broad range of scenarios tested, during the development and testing process, we identified a number of limitations that can be improved in the future:

Internet Connectivity Dependency

Event Sphere is currently an online application, requiring internet connectivity for almost all of its features, including searching and viewing events, registering for events, using QR codes to track attendance, sending notifications, as well as administrative tasks. The application is not usable in low- or no-signal areas. Although Firebase Firestore provides some offline capabilities, these were not fully set up in this version, so event information may not be available offline. This makes the app less useful in the context of university where network connectivity may be unreliable.

Android Platform Only

Event Sphere is currently only compatible with Android devices on Android 8.0+. The first release doesn't support iOS devices because of the resources required for platform-specific development and testing. This restricts the use of the app to students and teachers with iPhone devices. Although Flutter's cross-platform ability means that the iOS version can be developed without rewriting the code, testing, Firebase configuration and App Store deployment issues must be resolved to release the iOS version.

QR Scanner Performance Under Low Lighting

During testing, there was a slight performance degradation of the QR code scanner module in very low lighting conditions. When the event space was dimly lit, some students needed to scan their QR code multiple times for the code to be captured by the camera. The mobile scanner package works well under normal lighting conditions, but the current version does not automatically turn on

the phone's flash or automatically adjust the exposure of the camera to adapt to low light conditions. As a result, it may cause slight delays in marking attendance for the event.

Limited Offline Functionality

While Event Sphere includes a basic offline caching layer using Hive local storage (CacheService) and a connectivity-aware action queue (OfflineService), the offline functionality is limited to caching previously loaded event data. Users cannot create new events, register for events, or perform QR attendance scanning while offline. Full offline mode with complete read-write synchronization remains a future enhancement.

Manual Fallback for QR Attendance

The existing system relies on Faculty to manually check and record attendance in the Faculty dashboard in the case of the QR code scanner or device camera not working on the event day. This creates a need for Faculty intervention for special cases of student attendance and may slow down the check-in process. The implementation of an alternative system such as an attendance PIN or manual attendance confirmation within the app would help to alleviate this.

7.3 Future Improvements

A number of improvements have been identified that will greatly enhance the functionality, user base and value of Event Sphere in future iterations:

iOS Platform Support

Extending Event Sphere to iOS using Flutter's cross-platform capabilities. This involves configuring Xcode build settings, Firebase iOS SDK integration, Apple Push Notification Service setup, and App Store submission compliance with Apple's Human Interface Guidelines.

Advanced AI Integration

Replacing the current rules-based chatbot with a AI-powered rules-based-powered assistant capable of natural language understanding, personalized event recommendations based on attendance history, and intelligent event summarization for quick user consumption.

Multi-Language Support

Implementing Flutter's localization framework (flutter_localizations, intl) to support Urdu, Arabic, and Chinese alongside English, with right-to-left layout support for Urdu and Arabic.

Payment Integration

Integrating Stripe or JazzCash payment gateways for paid event ticketing, with automated receipt generation, refund processing, and expense report integration.

Advanced Analytics Dashboard

Expanding the admin dashboard with predictive analytics using historical attendance patterns, real-time event popularity heatmaps, automated engagement reports, and exportable BI-style visualizations.

Calendar Integration

Implementing native calendar synchronization to add registered events to Google Calendar or Apple Calendar with automated reminders and schedule conflict detection.

Web Application Version

Deploying a Flutter Web companion application using Flutter's web rendering engine for desktop access, enabling broader reach for event browsing and administrative tasks.

Virtual Event Hosting

Implementing an in-app virtual event hosting module with live streaming using WebRTC, interactive Q&A sessions, virtual breakout rooms, and real-time polls during live events to support hybrid physical-virtual event formats.

References

- [1] Google, "Flutter Documentation," Flutter. [Online]. Available: <https://docs.flutter.dev/>. [Accessed: June 2025].
- [2] Google, "Firebase Documentation," Firebase. [Online]. Available: <https://firebase.google.com/docs>. [Accessed: June 2025].
- [3] Supabase Inc., "Supabase Documentation," Supabase. [Online]. Available: <https://supabase.com/docs>. [Accessed: June 2025].
- [4] Google, "Google Maps Platform Documentation," Google Developers. [Online]. Available: <https://developers.google.com/maps>. [Accessed: June 2025].
- [5] Google, "Firebase Cloud Messaging Documentation," Firebase. [Online]. Available: <https://firebase.google.com/docs/cloud-messaging>. [Accessed: June 2025].
- [6] Google, "Cloud Firestore Documentation," Firebase. [Online]. Available: <https://firebase.google.com/docs/firestore>. [Accessed: June 2025].
- [7] DENSO WAVE INCORPORATED, "QR Code.com - QR Code Essentials," [Online]. Available: <https://www.qrcode.com/>. [Accessed: June 2025].
- [8] R. Sandhu, E. Coyne, H. Feinstein and C. Youman, "Role-Based Access Control Models," IEEE Computer, vol. 29, no. 2, pp. 38-47, Feb. 1996.
- [9] Flutter Team, "State Management in Flutter," Flutter Documentation. [Online]. Available: <https://docs.flutter.dev/data-and-backend/state-mgmt>. [Accessed: June 2025].
- [10] K. Beck et al., "Manifesto for Agile Software Development," Agile Alliance, 2001. [Online]. Available: <https://agilemanifesto.org/>. [Accessed: June 2025].